



Hellenic Mediterranean University

School of Engineering

Department of Electrical and Computer Engineering

Stock market prediction using machine learning and neural network methods

Undergraduate Thesis: (B.Eng.) Informatics Engineering (Software engineering)

Konstantinos Papi (Student Registration Number TP4720)

Supervisor: Dr. Konstantinos Karampidis

Acknowledgements

«Στην οικογένεια μου που με στήριξε, και πάνω απ' όλα στην μητέρα μου...»

Abstract

In the field of finance and predictive analysis, various methods have been used to predict the stock market for positive returns. In this thesis, we test the deployment of machine learning and neural network models to make accurate predictions on the stock market. We use timeseries data which consist of features that gives us the exact stock prices within a window of 4 years. After the collection of the data, we move to data preparation in an appropriate format to feed the models. We then deploy 4 machine learning models and 4 neural network models to compare both categories for performance on the stock prices without the danger of overfitting or underfitting. For the evaluation we use the RMSE metric to measure the success among the machine learning and the neural network models. The results for the evaluation metrics of our models ranged from 0.94 to 14.26 RMSE.

Περίληψη

Στον τομέα των χρηματοοικονομικών και την προβλεπτικής αναλυτικής, διάφοροι μέθοδοι έχουν χρησιμοποιηθεί για την πρόβλεψη μετοχών με θετικό αντίκτυπό κέρδους. Σε αυτή την πτυχιακή εργασία θα δοκιμάσουμε την χρησιμότητα των μοντέλων μηχανικής μάθησης και νευρωνικών δικτύων για την ακριβή πρόβλεψη των μετοχών στο χρηματιστήριο. Τα δεδομένα μας είναι χρονοσειριακής μορφής και περιέχουν χαρακτηριστικά σχετικά με της μετοχές σε δεδομένη χρονική περίοδο τεσσάρων ετών. Μετά την απόκτηση των δεδομένων προχωράμε σε μεθόδους προ-επεξεργασίας για την προετοιμασία των δεδομένων στους αλγόριθμους. Μετέπειτα θα προχωρήσουμε στην εκπαίδευση με 4 αλγόριθμους μηχανικής μάθησης και με 4 αλγόριθμους νευρωνικών δικτύων. Για την μέτρηση της επιτυχίας χρησιμοποιούμε την μετρική RMSE που μας βοηθάει να έχουμε αντικειμενική εικόνα για τα αποτελέσματα μεταξύ των δυο κατηγοριών μεταξύ τους. Τα αποτελέσματα έχουν εμβέλεια από 0.94 RMSE έως και 14.26 RMSE.

Table of Contents

Acknowledgements	i
Abstract	ii
Περίληψη.....	iii
Table of Contents.....	iv
List of Tables	vi
List of Figures	vii
Chapter 1. The Stock Market.....	1
1.1 Introduction	1
1.1.1 What is the stock market?	1
1.1.2 History of the stock market and the modern stock market.....	1
1.2 The nature of the problem	2
1.2.1 Traditional Stock Evaluation Methods	2
1.2.2 The impact of Computer Science, Computational Intelligence and Applications on Finance	5
1.2.3 Modern methods for stock evaluation and forecasting.	5
1.3 Problem statement & Structure of thesis	6
Chapter 2. Artificial intelligence	7
2.1 What is Artificial Intelligence	7
2.2 Types of Artificial Intelligence	8
2.2.1 Machine learning	8
2.2.2 Knowledge Based systems	10
2.2.3 Neural networks	11
Chapter 3. Algorithmic Analysis of ML and NN Models	12
3.1 Machine learning algorithms.....	12
3.1.1 Linear Regression.....	12
3.1.2 Decision Trees Algorithm and Random Forest	13
3.1.3 Gradient Decent.....	15
3.1.4 XGBoost.....	15
3.1.5 SVM and SVR.....	17
3.2 Artificial Neural Network and Deep Learning	19
3.2.1 Perceptron.....	19
3.2.2 Multi-Layer Perceptron	20

3.2.3 Convolutional Neural Network	22
3.2.4 Long - Short-Term Neural Network	25
Chapter 4. Related work	28
4.1 Machine Learning Methods.....	28
4.2 Neural Network methods.....	29
4.3 Other Methods	31
Chapter 5. Implementation methodology	33
5.1 Machine learning workflow	33
5.2 Dataset	33
5.3 Data preparation	34
5.4 Evaluation metrics	34
5.5 Hardware	34
5.6 Packages and Frameworks	35
5.7 Hyperparameters and models	35
Chapter 6. Results & Conclusions.....	37
6.1 Experimental results	37
Chapter 7. Conclusions and future work	41
Bibliography & References	42
Appendix	48

List of Tables

Table 1, System hardware.....	35
Table 2. Parameters & Hyperparameters.....	36
Table 3. Root Mean Squared Errors for the Machine Learning models.....	37
Table 4. Root Mean Squared Errors for the Neural Network models	37

List of Figures

Figure 1 .Support and Resistance Pattern.....	4
Figure 2. Head and Shoulders	4
Figure 3.Categories of Artificial Intelligence Source: [68].....	7
Figure 4. (a) The height and weight of some people. (b) A possible clustering using K=2 clusters. Figure generated by kmeansHeightWeight. Source :[14].....	9
Figure 5. Knowledge Based System under the hood.....	11
Figure 6. Simple linear regression, Source: [22].....	12
Figure 7. Decision trees.....	13
Figure 8. Random forests with the bagging method including decision trees. Source: [66]....	14
Figure 9. XGBoost and the linear processes that happening with the gradient boost source: [66]	16
Figure 10. Hyperplane used to classify data on a 3-D dataset source: [65]	17
Figure 11. Linear function used to classify data on a 2-D dataset with the margin of the linear classifier and the ϵ deviation source: [33]	18
Figure 12. McCulloch-Pitts Single – Layer Perceptron model, Source: [58]	19
Figure 13. Multi-Layer Perceptron model, Source: [58].....	21
Figure 14. Convolution operation between a kernel (a) function and an input matrix (b) producing the output, Source:[61].....	22
Figure 15. Average Pooling example, Source:[40].....	23
Figure 16. Max Pooling example, Source:[40]	24
Figure 17. Convolutional Neural Network Architecture, Source:[63]	25
Figure 18. LSTM Architecture, Source:[62]	26
Figure 19. Machine Learning Workflow	33
Figure 20. Actual vs Predicted values from MLP model	38
Figure 21. Actual vs Predicted values from LSTM model.....	39
Figure 22. Actual vs Predicted values from CNN model	39
Figure 23. Actual vs Predicted values from CNN model.....	40

Chapter 1. The Stock Market

1.1 Introduction

The purpose of this chapter is to introduce the concepts and the theoretical framework with which we will deal with in our research. We introduce the field of stock market, the history of the stock market and types of stock market predictions methods. Finally, we will define the nature of the problem by explaining the importance of stock prediction task.

1.1.1 What is the stock market?

The stock market is a financial marketplace where economic transactions take place, including the buying and selling of stocks or shares, which represent ownership in a company. A stock, also referred to as a security, is listed on stock exchanges such as the New York Stock Exchange (NYSE), allowing investors to trade it. Stock markets play a crucial role in a country's economy. The price of a stock depends on the company's financial performance and reports. Additionally, stock prices can be influenced by political conditions, corporate strategies and innovations, and a country's overall economic stability. In a globalized environment where the stock market operates, uncertainty is inevitable [1].

1.1.2 History of the stock market and the modern stock market.

The origins of the stock market are mentioned back to the 16th century, an era of significant discoveries that played a critical role in the expansion of the modern stock market. During this period, Dutch, Spanish, and Portuguese explorers travelled to North and South America, extracting goods and bringing them back to their home countries. At this stage, these goods were sold, leading to the emergence of major economic trade. Investors in northwestern Europe, particularly in Amsterdam and Antwerp, gathered to participate in commerce and exchange products for investment purposes and not only. A major milestone happened in the early 1600s when the Dutch East India Company issued bonds and stocks to the public. By the 19th century, the modern stock market had taken shape, with the London Stock Exchange (LSE) becoming the primary financial hub for Europe and the New York Stock Exchange (NYSE) serving as the center for American markets. The stock market practices of the 18th and 19th centuries were not as efficient as they are today, but they marked the beginning of the financial system that would dominate the global economy for centuries. In the 21st century, technological advances have revolutionized stock trading. Traditional methods of market analysis have been

replaced by modern computational techniques, and with the rise of the internet, most stock market transactions now occur online. The integration of computer engineering, software engineering, and mathematics has further transformed the way people trade, shaping the highly digitalized financial markets of today [1].

1.2 The nature of the problem

As mentioned, the stock price can be affected by many factors, such as political influence, economic growth, news, and financial analysis of a specific stock. These events can have a major effect on the stock price. The main concern of every broker, investor, or trader is predicting the next trends in the market and managing the risk in their portfolios. Although academics from disciplines like mathematics, economics, physics, and computer science have proposed methods in forecasting stock market trends, this remains a very complex topic [2]. The combination of these two provides the necessary information to buy or sell a specific stock. However, forecasting is challenging due to intrinsic noisy environments and high volatility. The stock market operates in a stochastic environment and is characterized by unpredictable fluctuations [3].

1.2.1 Traditional Stock Evaluation Methods

According to [3] and [4], technical and fundamental analysis were the most popular and effective methods for stock forecasting and analysis in the 20th century. Technical analysis is employed for short-term advantages, whereas the timeframe of fundamental analysis concentrates on the long-term generation of returns. It is crucial for the investor to filter out the short-term noise from the stock data and to grasp the long-term effects of the company's stock. In order to ascertain intrinsic value, fundamental research focuses on gathering information from pertinent aspects influencing the market price. The law of supply and demand is an illustration of intrinsic value. Fundamental analysis often investigates the reasons behind market fluctuations. Conversely, technical analysis is particularly concerned with the examination of market behavior. Technical analysis, on the other hand, uses techniques or patterns that can show up on the chart while studying market behavior [5],[6]. The capacity of the charting approach to handle various temporal dimensions and highlight the advantages and disadvantages of the economy is its main strength. Among the well-known and identifiable patterns are Support and Resistance (Fig.1), Head and Shoulders (Fig.2), Rising Wedges, and

Falling Wedges. These are most frequently seen patterns in the stock market. Combining technical and fundamental analysis gives investors a comprehensive picture of a stock's potential and many other advantages. Although each method has drawbacks, their combination can handle important quantitative aspects of stock analysis. Beginning with fundamental analysis to assess the health of a company is essential, and then narrowing down the technical part with technical analysis is useful. This combination is usually employed to create a portfolio report with the risks and returns expected [5].



Figure 1 .Support and Resistance Pattern



Figure 2. Head and Shoulders

1.2.2 The impact of Computer Science, Computational Intelligence and Applications on Finance

The fields of computer science and engineering have a significant influence on our society and its future. The eminent mathematician and computer scientist Alan Mathison Turing presented the initial components of artificial intelligence. As stated in [3] due to limitations of hardware, Alan Turing's work, which involved teaching machines to learn through concepts like "punishments" and "rewards" was vital to the discipline of artificial intelligence. A new kind of groundbreaking hardware masterpiece appeared in the 1960s and 1970s because of numerous fields and efforts coming together to produce the era of the minicomputer [4]. Scientists got to experiment with addressing obstacles that were impossible to do with traditional methods of learning and research thanks to minicomputers. Programming languages were used to develop statistical models in the domains of economics and computational intelligence. The first hedge fund manager and mathematician to use linear and logistic regression for stock market prediction was John Simons. He became the first successful hedge fund manager and gained an advantage on Wall Street as a result. Through the use of platforms and hybrid systems, businesses and brokers have changed into a digital environment in the digital era. Following the .COM boom of the 2000s, an increasing number of businesses made investments in the Internet and saw a new, pioneering approach to trading through computers, making trading and investing in digital financial products transactions possible. Simulations got essential for portfolio analysis, integrating techniques from the fields of statistics and econometrics, as software and network engineering advanced and major implementations changed, particularly in the banking industry. Finally, key economic indicators would be available for statisticians and financial analysts to compute and display [1].

1.2.3 Modern methods for stock evaluation and forecasting.

Modern stock evaluation and forecasting methods have evolved significantly, incorporating advanced techniques from both traditional finance and modern technologies. Statistical techniques and Machine Learning approaches are the new way of trading with High Frequency Trading being the next norm in the stock market. Statistical methods, time series analysis and regression analysis, are one of the most used in the trading sector paired with machine learning approaches that gave birth to complex systems referred to as Expert Systems[5]. These approaches, which combine multiple machine learning methods, provide more accurate and reliable recommendations [6].

1.3 Problem statement & Structure of thesis

This thesis aims to **"apply and compare machine learning, neural networks and deep learning algorithms to learn and predict stock market trends."** Ultimately, the metrics that each algorithm achieves when applied to compared financial information will allow us to determine which model works better. The tools performance is compared across all datasets. A particular timeline date is used to pick the datasets and once features like start and end dates are added, the dataset is referred to as a time series dataset.

The structure of thesis is organized into seven chapters. In the 1st chapter we focus on the history of the stock market and more specifically, explains the stock market, what it is, and what economic impact it has. The 2nd chapter introduces artificial intelligence and explains the various categories that exist, describing each category with relevant examples. The 3rd chapter focuses on the machine learning and neural network algorithms used in the experiments conducted in this thesis, as well as the analytical mechanisms behind them, for a better understanding of data processing. In Chapter 4, we present the related work that has been done in the topic of the thesis. We categorize each related work by the solution that has been implemented to solve the same task. Chapter 5 shows the experiment workflow with all the discrete steps, also we mention the hardware resources and lastly, we reveal the libraries, parameters and hyperparameters that we use to our models. Chapter 6 presents to the reader the results of our experiments with quantitative metrics and qualitative explanations of the model's performance. Finally, in chapter 7, we highlight our conclusions about the work achieved and we propose some future work that can be done to make more efficient stock market predictions.

Chapter 2. Artificial intelligence

2.1 What is Artificial Intelligence

Artificial Intelligence has been through the lenses of the scientific community, particularly in the academic era of finance and risk investment. The development of AI models has the potential to analyze, forecast, and predict financial data that can lead to decisions with the least risk. In the academic world, a variety of research papers use artificial intelligence applications such as Knowledge-Based systems, Machine Learning and Neural Networks (Fig. 3) to create hybrid systems for stock prediction. The definition of artificial intelligence refers to the simulation of human intelligence from a computational machine[7][8]. There isn't a single, accepted definition of artificial intelligence, even though the academic community is conducting more research in this area [8]. The term was created by John McCarthy in 1955 and was defined as design and engineering of making intelligent machines [9]. Shortly, AI is a piece of software or possibly also hardware that humans program to complete complex tasks the user gives to it and through the right amount of data and human learning paradigms to execute the tasks with success.

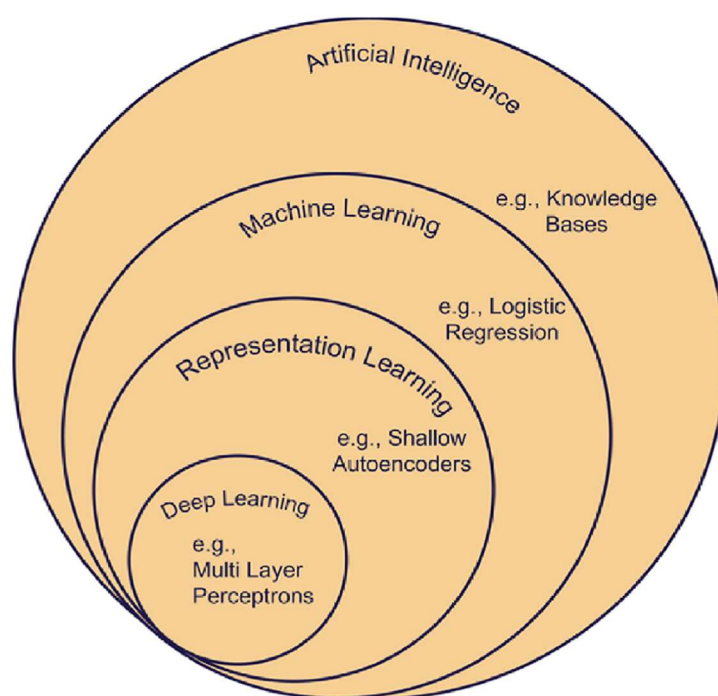


Figure 3.Categories of Artificial Intelligence Source: [68]

2.2 Types of Artificial Intelligence

As shown in (Fig.3) the categories of artificial intelligence are split into 3 main categories:

- Machine Learning methods
- Knowledge-Based Systems
- Neural Network Systems

The meta-analysis from Muhammad [10] et. al has shown that the most used methods for stock market predictions are the machine learning models such as Linear Regression (LR), Support Vector Machines (SVM), and Neural Network methods like Artificial Neural Networks (ANN), Long Short-Term Memory (LSTM), Deep Neural Networks (DNN), and so on. A study from Priyanka Tupe-Waghmare [11] shows that most researches in the topic of Stock market predictions started from 1998, and during 2013 to 2014 it skyrocketed with the number of new papers rising with exponential growth, after 2018 showed a decrease. Most of the papers that appeared in recent years focus mostly on Deep Learning methods, although machine learning methods are also effective. One of the earliest works P.D. et al. [12] introduced machine learning and neural networks in the field of stock market prediction.

2.2.1 Machine learning

Machine learning is a field of AI that is mostly implemented for performance through our input and definitions. Machine learning (ML) is a sequence of algorithms that can detect patterns in data and then use the uncovered patterns to predict future data or to classify potential data through the process of classification[13]. Formally in ML methods, the prediction is created as a function of the value of a hypothesis map whose arguments (inputs) are the features of a data point.

Human learning is considered one of the most complex systems. As we grow through experiences and time, new neurons are created and connected to create a path that can form our habits on autopilot. Usually, to form a habit or to learn in general, we can get information from two specific logical assumptions. The natural way of learning is by trial and error, which is the building block of our knowledge. The second method involved learning by other humans and their experiences. Their experiences can give us knowledge and new forms of rules and facts that can accelerate the learning process in any field of interest. Combining these methods can give us complete tools that can be used to learn effectively with practice and repetition.

Corresponding to AI, scientists and engineers have been replicating logical concepts to train similarly to human experiences, models that can give us the results that we desire.

Supervised learning is the form of learning that includes features that can give us the outcome that we want. The features include labeled data that can be used as a measurement for the correct prediction or classification that we desire from the model. From a technical perspective, the goal is to learn from inputs x to outputs y .

$$D = (X_i, Y_i)_{i=1}^N \quad (1)$$

In a simple version, each training input x_i is D-dimensional vector of numbers (equation 1), representing in our case the price of the stock and the y_i is our label feature that has the value that we want for the stock to have in N period [13].

Unsupervised learning includes the features of the data without any labeled feature (equation 2), the goal of this type of learning is the model to understand the nature of the data and to get the results that we want without any form of goal [14]. This is also called knowledge discovery in which the model can detect patterns that can be formed in which human pattern recognition is not so compelling to facilitate (Fig.4). One example of unsupervised learning is clustering formation from data into groups.

$$D = (X_i)_{i=1}^N \quad (2)$$

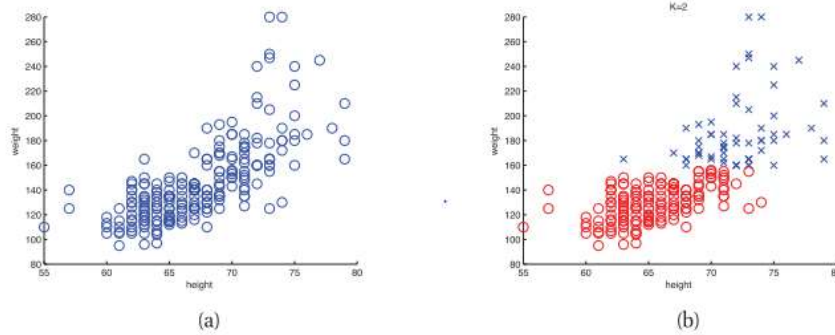


Figure 4. (a) The height and weight of some people. (b) A possible clustering using K=2 clusters. Figure generated by kmeansHeightWeight. Source :[14]

The third learning type, known as reinforcement learning, is the technique that involves the concepts that Alan Turing has mentioned in his paper about reward and punishment through the changes in signals that are applied mostly to neural network models [14]. In general

Reinforcement learning focuses on applications where the predictions from the previous methods are applied to predict future data that is not in the current time zone or field [15].

2.2.2 Knowledge Based systems

In the 1980's it was the first time that the paradigm changed to more human reasoning capabilities introducing the symbolic AI or so called "Expert systems" or Rule based systems. The idea is that symbols or rules are signature functions or facts that have some parameters or as they called relationships between the parameters [16]. Deductive computation is one of the main schools of thought for software development and the backbone of Logic programming [17]. The database or facts are structured through a relationship between the definition and the object such as:

male(john).female(sara).

father(john,sara).

daughter(sara,john).

which they describe that the John is the father of Sara and in order John to be the father of Sara the object John needs to be a Male and for the object Sara needs to be a Female to be daughter of John. Rules on the other hand are created to explain more complex facts between the knowledge base and the previous sub-rules that have or will be created. An example of rules is:

family(X,Y) : -father(X,Y),male(X),female(Y).

This rule explains that for a family to form one of the ways that can be formed is between the variable X to be a male, the variable Y to be a female and the X father of Y. Of course, we can add more rules to define the many variations that can be true to consider a family. The definition of logic programming states that a logic program is a list of facts and rules. More typically, a logic program is a finite set of rules formulated as definite clauses[17]. First order logic is applied and used from a theoretical standpoint that can navigate through the creation of the rules and the whole system itself (Fig. 5).

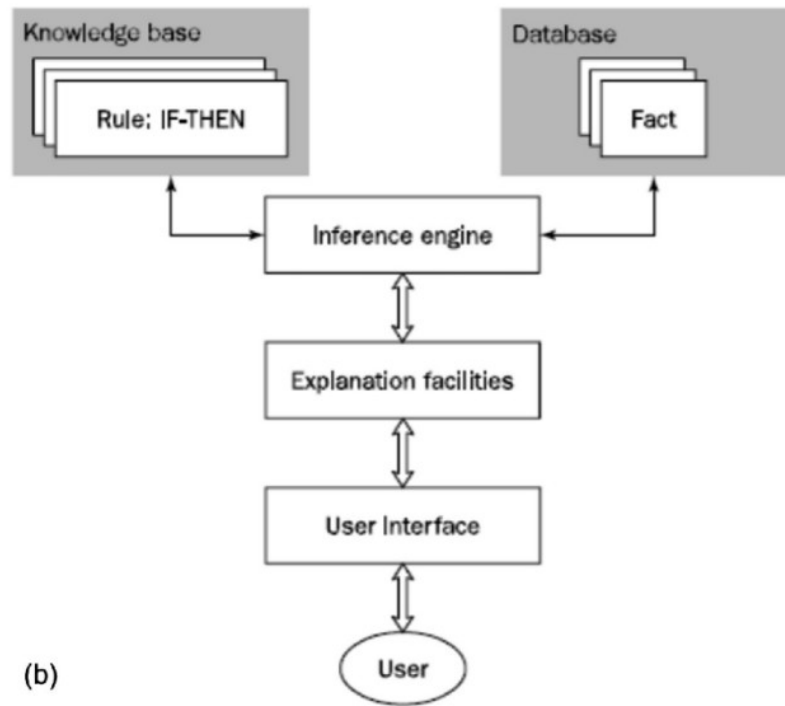


Figure 5. Knowledge Based System under the hood

Most of the systems that were developed are used in various endeavors, including Autonomous Anti-UAV Systems [18], educational purposes for STEM students engaged in Artificial Intelligence [19] and an exceptional work has been done in Natural Language Processing using prolog [20].

2.2.3 Neural networks

The evolution of neural networks has proliferated in science and engineering, making breakthroughs and creating products tailored to our desires. Neural Networks differ from machine learning, a complex way of networking between nodes generates a path that leads to the desired output. As neural network design can be described as an acyclic graph [21], Deep Learning is a form of neural network that has many hidden layers, making neural pathways more complex and deeper in processing. A Neural network usually contains a small number of hidden layers and does not include preprocessing models before or after the model. Usually, deep learning architecture includes complex systems and models that can process the outputs and then reenter them in a new hidden or end layer. There are many variations, and each plays a role in a specific problem that we want to solve.

Chapter 3. Algorithmic Analysis of ML and NN Models

3.1 Machine learning algorithms

This chapter will analyze various systems and algorithms with their mathematical definitions to explain what is working under the hood.

3.1.1 Linear Regression

Considering that the data points are vectors X and Y , linear regression is concept for predicting a value Y based on a single predictor variable X . It assumes a possible linear relationship (equation 3) between X and Y . Mathematically, we can explain linear regression as:

$$Y \approx \beta_0 + \beta_1 X + \beta_2 X_2 + \dots \beta_n X_n + \epsilon \quad (3)$$

For example, X represent Car Model Quality, and Y represent the sales.

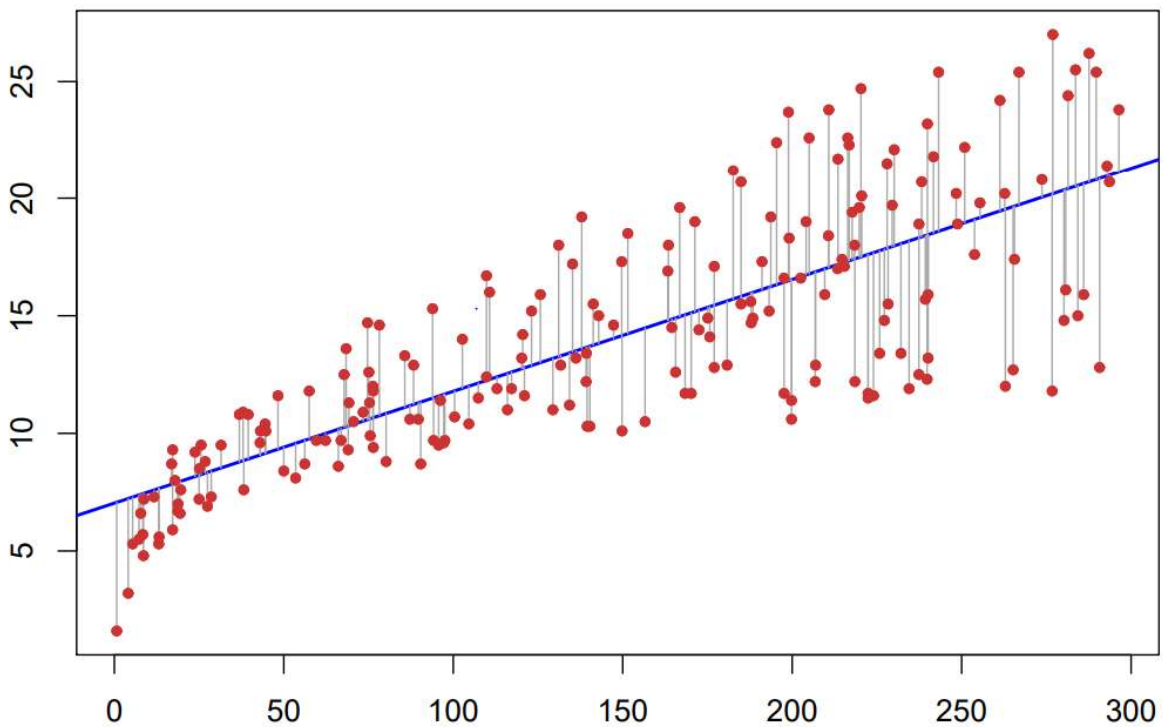


Figure 6. Simple linear regression, Source: [22]

In Fig. 6, we display a simple linear regression fit to the CMQ(X-axis) onto Sales (Y-axis) where $\beta_0 = 7.03$ and $\beta_1 = 0.0475$ [22]. The Y is the dependent variable, which corresponds to the predictions that we want to make, and the X is the independent variable, which

corresponds to the features that our model uses to model the Y . The β_i are the coefficients -also called weights- of our regression model. They are responsible for the learning part of our model during optimization. Lastly, the $\mathcal{E}$ represents the residual error that we can have in our model between our linear predictions and the true response. Fitting a linear regression model is all about finding the optimal coefficients that represent the predictions that we want.

3.1.2 Decision Trees Algorithm and Random Forest

Decision Trees (DT) are among the most used methods of machine learning and data science. The first decision trees (Fig. 7) were developed in the 1980s when technology was rising, and computer science communities were growing rapidly. This type of machine learning algorithm uses the term recursive partitioning known as “top-down induction of decision trees” [23]. A DT follows the process of computing the outcome Y (deterministic or probabilistic) of a function $f(x)$. The main idea of this computation follows the input X , where the input value is performed in a test to verify the output or close to our desired output. If it doesn’t meet our criteria, the outcome of each test determines the next test until the expected outcome is reached. In a more detailed manner, the first test is happening in the root node. The root node is the node that is the first hypothesis tested in which the test will happen to determine the split into 2 child nodes. Lastly the nodes that don’t produce more nodes are called leafs which usually gives us the possible outcome of our model. The leaf nodes contain subsets of the feature space. The depth of a decision tree can be defined by the max number of hops it takes to reach a leaf node.

X_1	X_2	X_3	Y
True	True	True	True
False	True	True	True
True	False	True	True
False	False	True	True
True	True	False	True
False	True	False	False
True	False	False	False
False	False	False	False

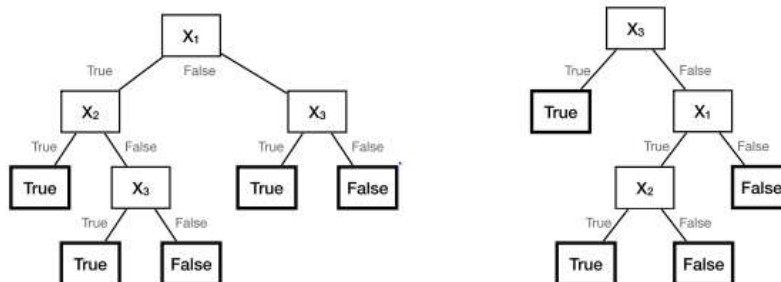


Figure 7. Decision trees.

In the 2000s a new schema started to make an appearance that will scale up the process of the decision trees. Random forests were proposed by Leo Breinman [24] and they measured substantial results in terms of classification and regression accuracy. The advantages that

random forests have is the control of large amounts of data and can overcome a phenomenon that can appear in a data world very often called overfitting. Overfitting means that the model has learned in a way that only applications and results apply to the training samples that we provided and no longer generalizes the overall problem that we need to solve [25] .

Random forests are a tree-based model (Fig.8) where classification or regression decision trees are trained on subsets of the data that we provide. Ensemble learning is a machine learning paradigm that involves combining multiple models or algorithms to produce a single, more accurate output. Bagging Aggregating is a method that is used to create the random forest paradigm. Every decision tree that is conducted has a specific percentage of the training set and the test set. All decision trees are combined into a larger random forest model, where each tree votes on the predicted value, and the value is selected by majority vote. [26]

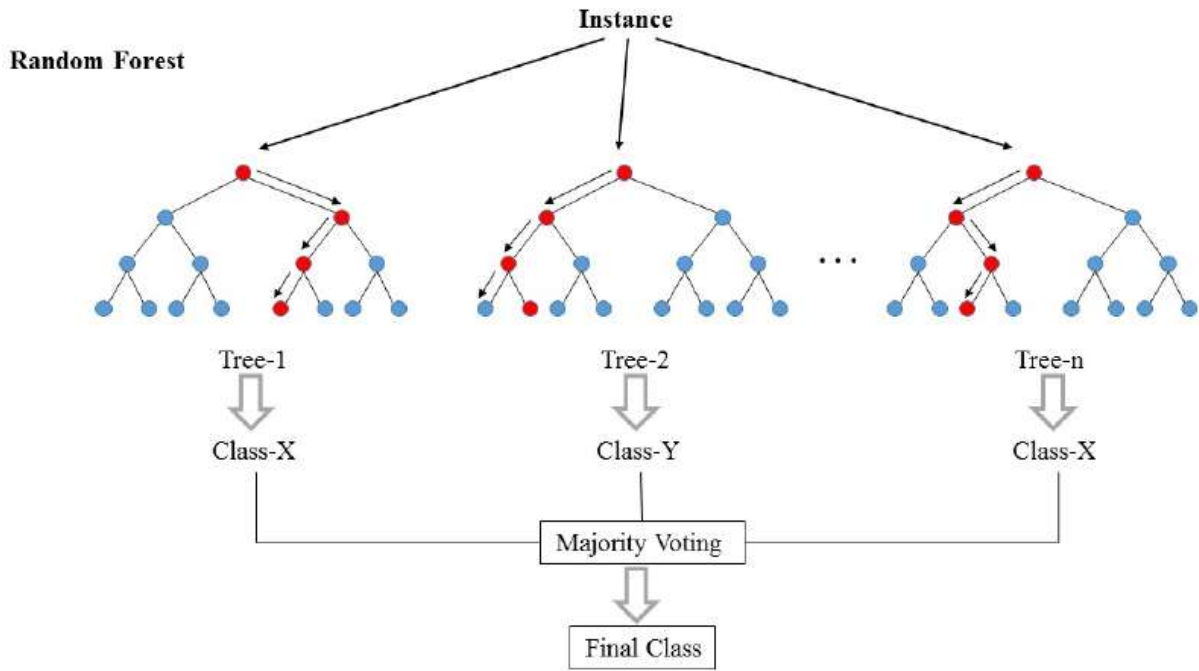


Figure 8. Random forests with the bagging method including decision trees. Source: [66]

In greater detail, the random forest workflow corresponds to m (equation 4) randomized decision tree as:

$$m_{M,n}(x; \theta_1, \dots, \theta_M, D_n) = \frac{1}{M} \sum_{j=1}^M m_n(x; \theta_j, D_n) \quad (4)$$

D is used to represent the different splits for from the training set θ for the decision trees that we want to be used. M is the number of trees in the forest and since M may be chosen randomly

large number (limited only by the computer power that we have) , it takes the form of a random number in a M (equation 5) infinite trees [27]:

$$m_{\infty,n}(x; D_n) = E_{\theta}[m_n(x; \theta, D_n)] \quad (5)$$

3.1.3 Gradient Decent

One of the most important topics related to the machine learning and neural networks community is the gradient descent (GD) function and optimization. In a greater detailed manner, the GD updates the values of a function to minimize the loss function of any differentiable function with its update step function to the parameter θ . The function $f(\theta)$ Exist in a domain of $\theta \in \Theta$ The goal is to find the minimum value $\theta^* = \operatorname{argmin}_{\theta \in \Theta} f(\theta)$. The optimization step function (equation 6)[28] :

$$\theta_{t+1} = \theta_t - a_t \nabla f(\theta_t) \quad (6)$$

- $\theta = \text{parameters}$
- $a = \text{learning rate}$
- $\nabla f(\theta_t) = \text{The gradient of the } (t)\text{th step}$

In other words, optimization is happening to find the optimal parameters in which the model weights get updated for better classification or regression. Many models use gradient descent to optimize machine learning algorithms that we discussed before and also a substantial number of neural network models and unsupervised learning algorithms are proposing different variations of gradient descent like stochastic gradient descent or ADAM for their specific needs or system combination in which vanilla gradient descent or other forms are combined with algorithms that enhance the quality and accuracy of a learning rate.

3.1.4 XGBoost

Extreme Gradient Boosting (XGBoost) (Fig. 9) combines two algorithms: decision trees and gradient descent. In other words, decision trees are the base learners, and the gradient descent works sequentially to reinforce the weak learners to perform better. XGBoost uses an ensemble learning method called Boosting, compared with Bagging from Random Forest boosting, it uses gradient descent to strengthen the weak learners by tuning their parameters slightly better each time to minimize the loss of the previous model. The final prediction is a weighted sum of all DT predictions [29], [30]. The XGBoost function calculates (equation 7)

$$obj(\theta) = TL(\theta) + R(\theta)$$

- $TL(\theta)$ stands for Training Loss
- $R(\theta)$ stands for Regulation term

The training loss calculates the quality of the prediction that the model provides (equation 8). At the same time, the Regulation term helps to maintain the model's complexity in a desired window of limits (equation 9), eliminating problems such as overfitting, which can lead to a less accurate model .

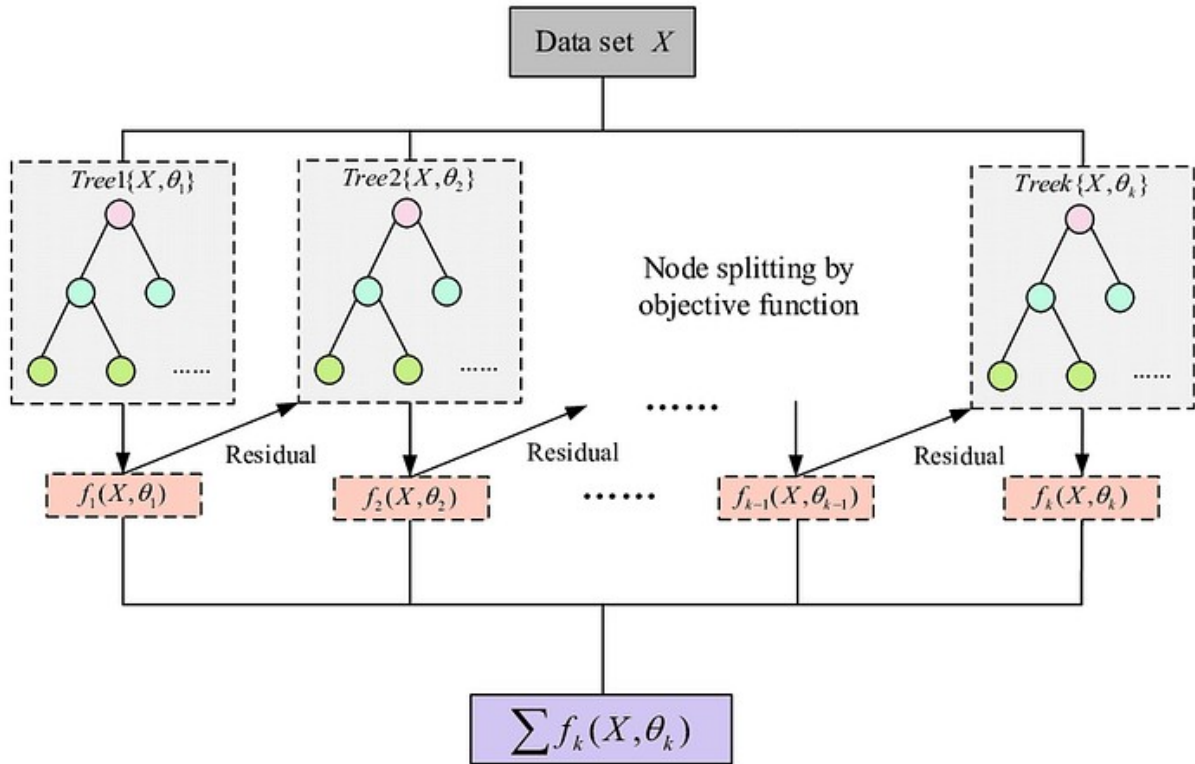


Figure 9. XGBoost and the linear processes that happening with the gradient boost source: [66]

The new $TL(\theta)$ The function can be described as Taylor's theorem (equation 8), in which the sum of the trees is stacked to perform the prediction. In Fig.9, the provided image visually shows the structure, so we can also see the pattern of a Taylor series.

- $TL(\theta) = \sum_{i=1}^n \left[m_i f_t(p_i) + \frac{1}{2} c_i f_t^2(p_i) \right]$ (8)
- m_i : first – order gradient(determines the magnitude)
- c_i : second – order gradient(updates step and ensures stability)
- $f_t(p_i)$: Prediction made by the newly added tree(t) for a given input(p_i)

- $R(\theta) = \alpha L + \frac{1}{2}\beta \sum_{j=1}^L w_j^2$ (9)
- $L = \text{Number of leaves in the decision tree}$
- $w_j = \text{Leaf score}$
- $\alpha = \text{L1 Regulation}$
- $\beta = \text{L2 Regulation}$

The XGBoost is very effective in prediction relative to random forests, with the condition of parameter tuning.[29]. The XGBoost algorithm has been used in medical datasets and cybersecurity systems to classify and detect server logs, medical images, and patient check-up histories [31], [32] . The XGBoost gives us an attractive promise for usage in stock market prediction.

3.1.5 SVM and SVR

Support vector machine (SVM) is a supervised machine learning method for classification and regression problems. SVM is part of the generalized linear classifiers and uses machine learning theory to create accurate results while avoiding over-fitting data [33].

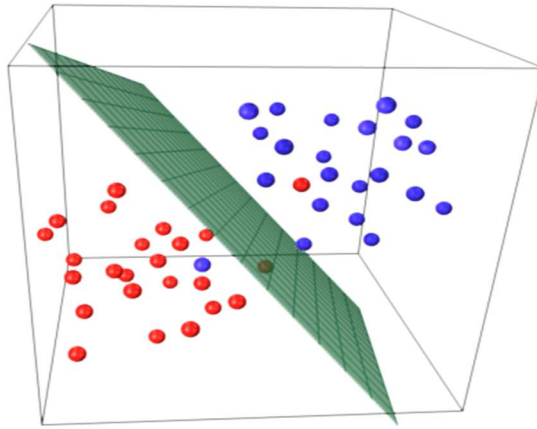


Figure 10. Hyperplane used to classify data on a 3-D dataset source: [65]

The difference between the SVM algorithm and other linear algorithms lies in the nonlinear kernel functions that distinguish high-dimensional feature data. This helpful tool can be transformed depending on our needs for classification or regression purposes [34]. An example of High dimensionality is pictured in (Fig. 10), where the dataset is classified into 2 clusters while we consider X, Y, and Z (3 dimensions) as input. The goal of the SVM is to find a function $f(x)$ or a linear classifier (hyperplane) that has at most ϵ deviation from the obtained target y_i and at the same time to be as flat as possible. In a more simplified version, we care

about the errors that don't exceed the ε . This linear classifier (Fig.11) is called a maximum margin linear classifier (equation 10) and represents the simplest kind of SVM the Linear Support Vector Machine (LSVM) model.[33]

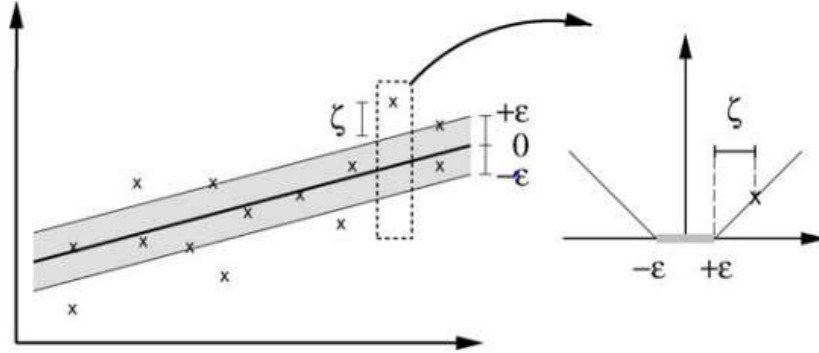


Figure 11. Linear function used to classify data on a 2-D dataset with the margin of the linear classifier and the ε deviation source: [33]

The margin classifier is defined by the:

$$\text{margin hyperplane} = \text{Argmin}(x) = \text{argmin} \frac{|xw+b|}{\sqrt{\sum_{i=1}^n w_i^2}} \quad (10)$$

For $\{(x_1, y_1), \dots, (x_n, y_n)\} \subset X * \mathbb{R}$ where the X represents the space of inputs, the x_n represents the input vectors, y_n represents the target vectors and n the total number of training examples, we can describe the linear function (equation 11) in the form of :

$$f(x) = \langle w, x \rangle + b, \quad w \in X, b \in \mathbb{R} \quad (11)$$

The hyperplanes that determine the classification are with a margin ε :

- $wx + b = 1$
 - $wx + b = 0$ (*Max margin hyperplane*)
 - $wx + b = -1$
- (12)

In the more real-world scenario in which data tends to have outliers, the type of margin classifier (equation 12) will determine their classification. Depending on the soft margin or hard margin classifier, the outliers may be excluded from the dataset or non-relevant. The difference between the hard and soft margins is that the soft margin allows us to consider vectors that are between the $-1 < \text{margin hyperplane} < 1$. The Hilber law gives us the formula to position the vector within the margin.

3.2 Artificial Neural Network and Deep Learning

This chapter will explain and analyze the neural network algorithms with examples and also define their mathematical functions to understand the processing behind the scenes. We will visualize some of the architecture in der to explain clearly the processing flow.

3.2.1 Perceptron

The first appearance of the perceptron model (Fig. 12) was introduced by Rosenblatt F [35] describing with mathematical functions the nature of biological senses while interacting with information or natural stimuli. Later on, the perceptron model created by McCulloch-Pitts [36] shows us a simplified version of the modern neural networks in which the modern A.I models are based on.

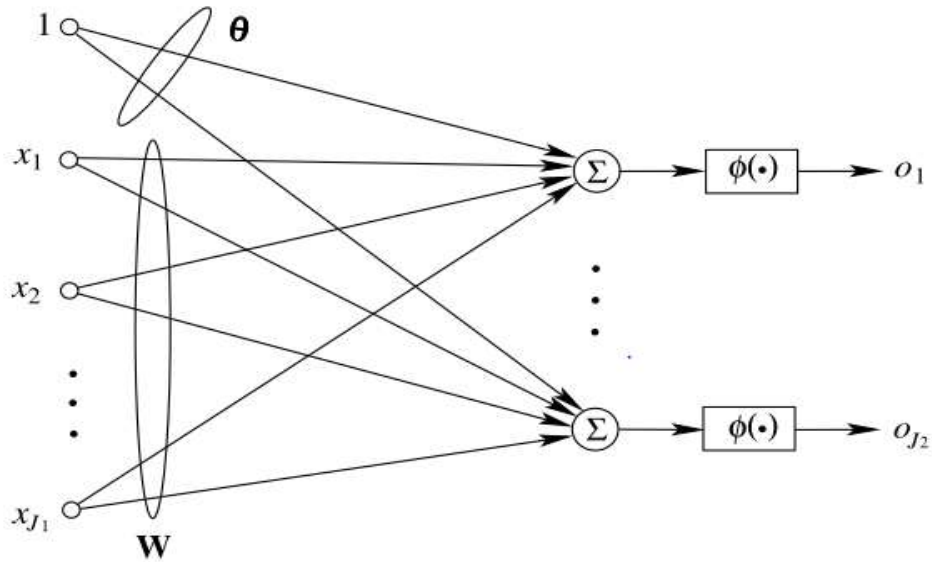


Figure 12. McCulloch-Pitts Single – Layer Perceptron model, Source: [58]

The perceptron consists of input vectors $\vec{x} = (x_1, x_2, x_3, \dots, x_n)^T$, they correlate with the data that we provide to the neural network, the weight vectors $\vec{w} = (w_1, w_2, w_3, \dots, w_n)$. The Bias (threshold) : θ { vector }. In general, a node of a neural network input (equation 13) is defined as:

$$Net = \vec{w}^T \vec{x} + \theta, \vec{o} = \varphi(Net) \quad (13)$$

The $\vec{o} = (o_1, o_2, o_3, \dots, o_n)^T$ Is the output vector, and φ_i is the activation function of the i_{th} Output neuron. The activation functions (equations 14,15,16,17) can determine which node the

input vector will pass through to the next node or give us the output. There are many activation functions, such as:

$$\varphi(\alpha) = \begin{cases} 1 & a \geq 0 \\ -1 \text{ or } 0 & a < 0 \end{cases}, \quad \text{Hard Limiter} \quad (14)$$

$$\varphi(\alpha) = \frac{1}{1 + e^{-\beta\alpha}}, \quad \text{Logistic} \quad (15)$$

$$\varphi(\alpha) = \tanh(\beta\alpha), \quad \text{Hyperbolic tangent} \quad (16)$$

$$\varphi(\alpha) = a, \quad \text{Linear} \quad (17)$$

The most significant advantage of neural network algorithms is the Perceptron learning algorithm (equations 18,19,20) [37], which requires them to update their weights to correct the results in each repetition. The repetitions are called epochs; the larger number epochs, the better-trained the model will be. The Perceptron learning algorithm in which the neural networks learn is the following:

$$o_{t,j} = \begin{cases} 1, & \text{when } net_{t,j} > 0 \\ 0, & \text{otherwise} \end{cases} \quad (18)$$

$$e_{t,j} = y_{t,j} - o_{t,j} \quad (19)$$

$$\begin{cases} w_{ij}(t+1) = w_{ij}(t) + \eta x_{t,i} e_{t,j} \\ \theta_j(t) + \eta e_{t,j} \end{cases} \quad (20)$$

3.2.2 Multi-Layer Perceptron

Complexity increases as problems transition from linear to non-linear. While the perceptron model excels at solving linear problems, a new approach has emerged for non-linear challenges: the Multi-Layered Perceptron model (Fig.13), which effectively addresses these issues. The multilayered perceptron is an extension of the perceptron models, and one of the significant differences compared to the perceptron. The Backpropagation algorithm, introduced by Frank Rosenblatt in 1964, was the foundational mechanism on which the MLP networks are based. The development of differential and non-linear activation using the chain rule in Backpropagation made learning neural networks more effective. Backpropagation is the most used algorithm to train neural networks by backpropagating the gradient through each layer. This made efficient learning, thus creating deep neural networks [36].

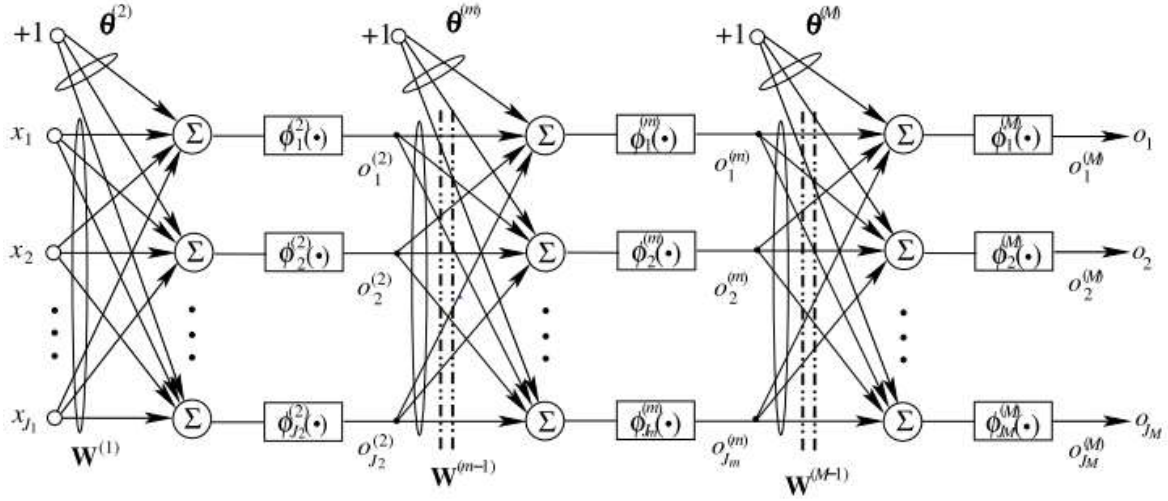


Figure 13. Multi-Layer Perceptron model, Source: [58]

The MLP network consists of M layers and each layer m has J_m Nodes. All the input nodes exist in the first layer of the model. The system contains weights $W^{(m-1)}$ that connect the nodes in each layer $(m+1)$ to layer m , the bias $\theta_l^{(m)}$ in each node, the output nodes $o_l^{(m)}$ and the activation function in each output node $\phi_l^{(m)}(\cdot)$. The network's output (equation 21) for the sample p is as $\vec{y}_p$:

$$\vec{net}_p^{(m)} = [W^{(m-1)}]^T \vec{o}_p^{(m-1)} + \vec{\theta}^{(m)} \quad (21)$$

We can see a similar structure of the single perceptron with the main difference, as we mentioned, the backpropagation learning algorithm. The backpropagation is referred from (equation 22,23,24,25) .

$$\partial W_i = - \left((W_{i+1}^T \delta m_{i+1}) \odot \phi'(m_i) \right) y_{i-1}^T \quad (22)$$

$$\delta z_{i+1} = \frac{\partial L}{\partial z_{i+1}} \quad (23)$$

$$y_i = \phi(z_i) \quad (24)$$

$$z_i = W_i y_{i-1} + b_i \quad (25)$$

3.2.3 Convolutional Neural Network

Convolutional Neural Network (CNN) models are a great example of complex yet effective neural network for pattern recognition or prediction tasks. They are like traditional ANNs, meaning that neurons are self-optimized through learning. The difference relies on convolution. Convolution is a mathematical computation that generates a function (z) that expresses how the features of (f) are transformed by the other (g) [38].

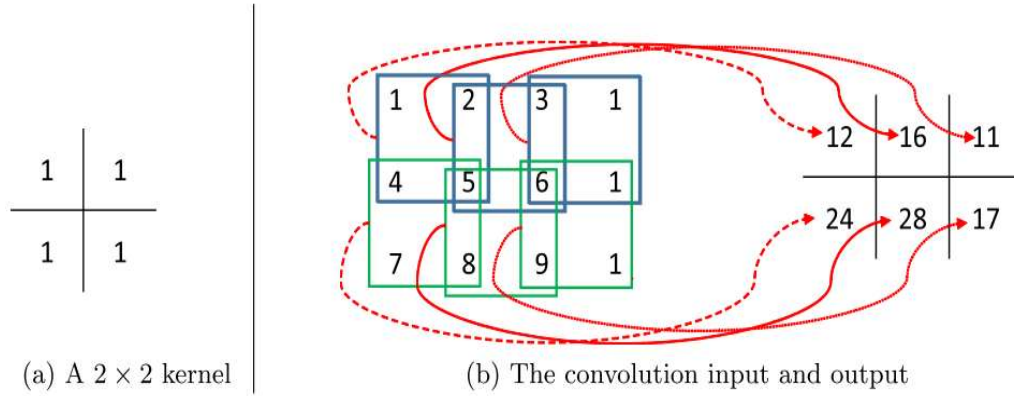


Figure 14. Convolution operation between a kernel (a) function and an input matrix (b) producing the output, Source:[61]

The mathematical description of convolution can vary based on the dataset's dimensions; thus, the type of dataset will determine the convolution process's output. In (Fig. 14), we perform a convolution on a 2-D dataset, typically an image, where the data represents the pixel values of a photo. If we place the kernel over the top left corner of the input, the convolution results will be as follows: $1 \times 1 + 1 \times 4 + 1 \times 2 + 1 \times 5 = 12$. This operation continues for the subsequent windows, applying the kernel to the input to obtain the output. In greater detail, the mathematical description (equation 26) of the 2-D convolution layer is as:

$$S(i,j) = \sum_{m=0}^2 \sum_{n=0}^2 I(i+m,j+n)K(m,n) \quad (26)$$

The $S(i,j)$ is the output at (i,j) in the feature map produced after convolution. It represents the result of applying the filter at that location. $I(m,n)$ is the input value in the input feature map at position (m,n) . The $K(m,n)$ describes the kernel (filter). This matrix contains the weights that the network learns to detect specific features. The next step or layer that follows the convolution

layer is called the pooling layer. The pooling layer helps to reduce the dimensionality of the features and reduce the computational complexity for upper layers by removing connections between convolutional layers. The layer uses a down-sampling method for reducing the number of parameters or weights in order to reduce the computation and the control of model overfitting or not. The main idea is to extract only useful information and discard unrelated details[39]. There are many pooling layer methods, while the most used ones are the Average Pooling operation (equation 27)(Fig.15) and the Maximum Pooling operation (equation 28)(Fig.16) . Of course there are many as such Stochastic Pooling ,LP Pooling, Mixed Pooling , each of them have their own specific use cases but we will focus on the Average and the max one. An average pooling calculation (Fig.15) the average of a 2x2 Pooling window (C) is used to produce the outcome on the Pooled feature map.

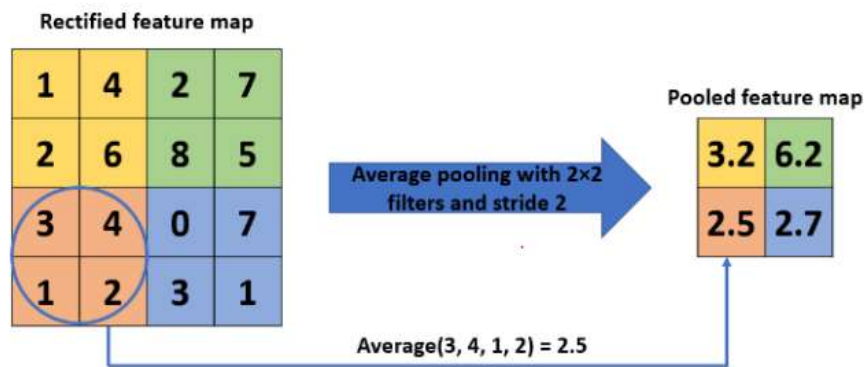


Figure 15. Average Pooling example, Source:[40]

The average pooling is described as[40]:

$$APFM = \frac{\sum C}{|C|} \quad (27)$$

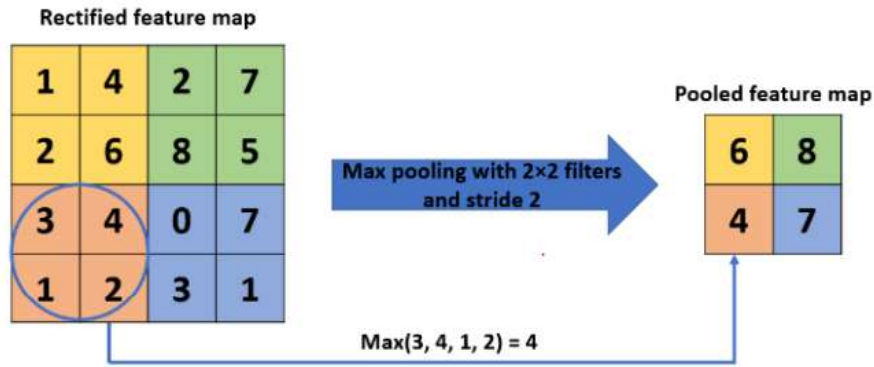


Figure 16. Max Pooling example, Source:[40]

As for the Max pooling the function is described as [40]:

$$MPFM = \text{MAX}(C) \quad (28)$$

Finally, the Activation layer and the Fully Connected layer serve as the points where classification or regression occurs, allowing the network to learn from its previously processed inputs. The Activation layer changes the linear function of inputs, which is quite restricted in its ability to learn from complex data, into a non-linear function. This non-linearity enables the network to learn and identify intricate mappings from the data[41]. The Fully Connected layer is a neural network like an MLP for example that processes the data extracted from the Convolution layer and the pooling ones to perform high-level reasoning and decision-making features. Connecting the FC layer the network can learn relationships and dependencies within the data that has been trained on. The activation function is applied into the FC layer to create the desired network outputs[42].

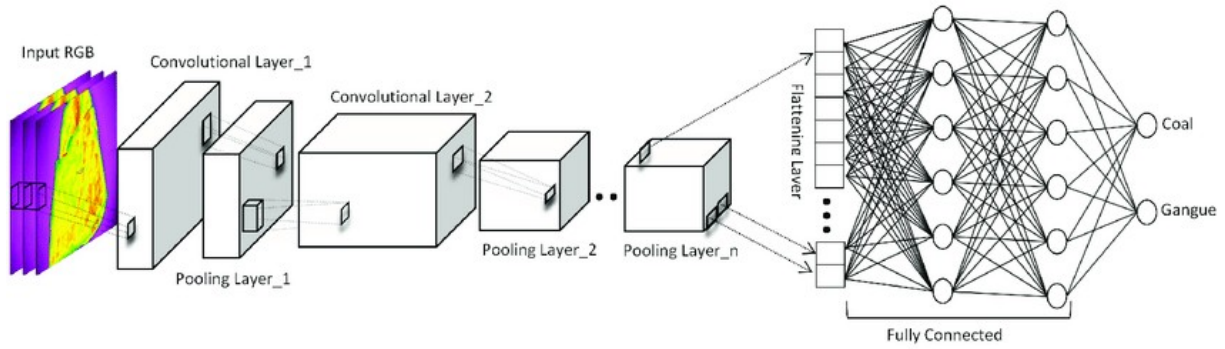


Figure 17. Convolutional Neural Network Architecture, Source:[63]

Overall, the CNN model can have many layers (Fig. 17) where the data can be processed. More complex models involve patterns of convolution layers passing through a pooling layer, followed by another convolution layer. The LC layer can also be modified for better accuracy and predictions, featuring various layers within the fully connected one. Different activation functions and the output layer create the final model. Tuning and finding the optimal epochs, convolution methods, and pooling methods for specific tasks is key to creating the optimal network.

3.2.4 Long - Short-Term Neural Network

Long-Short Term Memory (LSTM) neural networks were introduced by Hochreiter & Schmidhuber in 1997 [43], addressing the problems associated with Recurrent Neural Networks (RNN). Error signals flowing backward in time tend to either blow up or vanish, complicating the backpropagated error. After numerous calculations, the optimization of the weights often struggled to update effectively due to significant changes in the network without a prior context for learning, the neural network struggled to understand long periodic patterns. The LSTM is designed to overcome these issues with error backflow. This is achieved by using an efficient, gradient-based algorithm. The vanilla LSTM architecture (Fig. 18) consists of recurrently connected networks named memory blocks. The main idea behind the memory block is to sustain its state over time and control the information flow through nonlinear gating units.

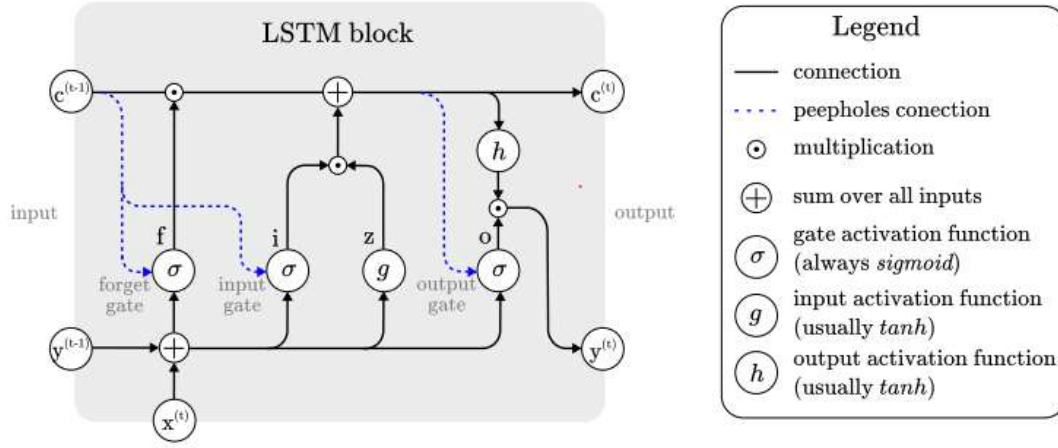


Figure 18. LSTM Architecture, Source:[62]

To describe the LSTM model, we first need to assume a network of N processing blocks and M inputs. Block input: This section focuses on updating the block input, which combines the current input $x^{(t)}$ and output of that LSTM unit $y^{(t-1)}$ and the cell value from the last iteration. The mathematical description (equation 29) to understand the linearity between the blocks and how the LSTM works under the hood is described as:

$$z^{(t)} = g(W_Z x^{(t)} + R_Z y^{(t-1)} + b_Z) \quad (29)$$

Where W_Z and R_Z are the weights associated with $x^{(t)}$ and $y^{(t-1)}$ respectively and the b_Z stands for the bias weight vector.

Input Gate: we update the input gate (equation 30) that combines the current input $x^{(t)}$, the output $y^{(t-1)}$ and the cell value $c^{(t-1)}$ in the last iteration.

$$i^{(t)} = \sigma(W_i x^{(t)} + R_i y^{(t-1)} + p_i \odot c^{(t-1)} + b_i) \quad (30)$$

The $\odot$ tell us that we use the multiplication of two vectors, W_i, R_i and p_i are the weights associated with $x^{(t)}$, $y^{(t-1)}$ and $c^{(t-1)}$. The b_i represents the bias vector.

Forget Gate: This step determined which information should be removed from its previous cell states $c^{(t-1)}$ (equation 31). The activation values $f^{(t)}$ of the forget gates at time step t are calculated based on the current input $x^{(t)}$, the outputs $y^{(t-1)}$ and the state $c^{(t-1)}$ of the memory cells at the previous time step $(t - 1)$, the connections and the bias terms b_f of the forget gates.

$$f^{(t)} = \sigma(W_f x^{(t)} + R_f y^{(t-1)} + p_f \odot c^{(t-1)} + b_f) \quad (31)$$

Cell: This block (equation 32) computes the cell value which combined the block input $z^{(t)}$, the input gate $i^{(t)}$ and the forget gate $f^{(t)}$ values, with the previous cell value :

$$c^{(t)} = z^{(t)} \odot i^{(t)} + c^{(t-1)} \odot f^{(t)} \quad (32)$$

Output Gate: This step calculates the output gate (equation 33), which combines the current input $x^{(t)}$, the output of that LSTM unit $y^{(t-1)}$ and the cell value $c^{(t-1)}$ in the last iteration.

$$o^{(t)} = \sigma(W_o x^{(t)} + R_o y^{(t-1)} + p_o \odot c^{(t)} + b_o) \quad (33)$$

Block output: We calculate the block output (equation 34), which combines the current cell value $c^{(t)}$ with the current output gate value:

$$y^{(t)} = g(c^{(t)}) \odot o^{(t)} \quad (34)$$

The advancement of LSTM model gave an opportunity for researchers to hold short term and long term memory within the model to make more accurate real-world models. Cases that involve LSTM model is Time series Prediction, Natural Language Processing (NLP), Sentiment Analysis, Image and Video Captioning, Computer Vision and Text Recognition.

Chapter 4. Related work

This chapter reviews the approaches that deploy computational intelligence to the regression process. We sorted these approaches into three categories:” Machine learning methods”, “Neural Network Methods” and “Other methods” including ML & NN for combined approaches and others that couldn’t fit into the first two categories.

4.1 Machine Learning Methods

Mehak Usmani et. al[44] used statistical models like Auto Regressive Integrated Moving Average (ARIMA), Vector Auto Regression (VAR), Vector Moving Average and Vector Auto Regressive Moving Average (VARMA) and also Deep Learning models like Multi-Layered Perceptron (MLP), Gated Recurrent Unit (GRU), Long Short-Term Memory and Generative Adversarial Network (GAN). The results showed the importance of preprocessing in which the data after a preprocessing phase gives the models a better score. In their work they used the Root Mean Squared Error (RMSE) as a measurement of scoring. The higher the score the better results. After the training phase the best results overall were achieved with ARIMA for the statistical models, SVR for the machine learning algorithms and on the DL algorithms the Bidirectional LSTM with GAIN Imputations scoring 10.028 RMSE.

Zhao [45] focused Linear regression for forecasting stocks. The dataset that was used was Samsung Electronics Stock Historical Price from Kaggle from 2019 to 2024. Data preprocessing was performed to eliminate missing values and the selected features were closing, opening, highest, and low price. The data was randomly selected with the ratio for training set being 80% and the rest 20% for the test set. The results stated that linear models could understand basic trends of the stock market, but in also had lower prediction accuracy when the stock market fluctuated.

Panwar et .al[46] used LR and SVM to predict stock trends. For the collection of data, they used a web scrapping method to web scrap yahoo finance. The results proved that LR had an accuracy of 98.76% compared to the SVM which had an accuracy of 94.32%.

Biswas et .al [47] proposed a method of stock market price prediction using LSTM, XGBoost, LR, and the statistical model Moving Average (MA). The dataset was retrieved from the Dhaka Stock Exchange and it contains 236 records which include the features Date, Opening, Low, Closing price and Adjacent close. The results stated that LSTM model scored 0.635 MAPE, XGBoost 1.114 MAPE, Linear Regression 1.287 MAPE, and Moving Average

1,297 MAPE making the LSTM model perform better than the rest and proving that most of the models could predict stock prices and learn the trends.

Kelany et al. [48] proposed a way to test the LSTM model, LogR and RF for its efficiency. The dataset contained stock with 3 categories including the risk i.e., Low risk, Medium risk and High risk. In general, the predictions stated that overall, the LSTM provided better scoring and accuracy compared to LogR and RF. The study concluded that the dataset timeline played a significant role in the training of the LSTM model. The 15 years data of the stocks made the model to have more accurate RMSE compared to the 5 years' timeline.

Henrique et al. [49] used a SVR model focusing in short-term forecasting while the model was up-to-the minute prices. The results showed that in total RMSE and MAPE showed slightly better results in the 2-year period rather than in the 3-month period in indicated better learning with larger dataset. The up-to-the-minute training showed also more accurate results rather than the fixed.

Jin [50] proposed statistical and deep learning models including LSTM, ARIMA, SVR, GRU and Holt's Linear Exponential Smoothing (Holt's LES). The dataset contains stock information for the S&P 500 index and SSE index for 1 day, 10 days, and 30 days. The results showed that the LSTM performed better all over across timelines, ARIMA and SVR models displayed moderate results. The Holt's LES performed better on 1 day and 10-day horizons and struggles to perform in larger timelines and lastly for the GRU model also struggled to perform well in 30-day timeline. The error percentage was calculated with RMSE, R^2 , MAPE, and MAE.

Wu et al. [51] proposed a method of hybrid deep learning algorithm called Triple Q in which the algorithm understand the gains and the losses as reward and so adapts to the changes. They compared the Triple Q algorithm with terms of capital returns. The proposed model performed statistically better in terms of returns.

4.2 Neural Network methods

Taewook Kim et al. [52] introduced hybrid models including the Convolutional Neural Network (CNN) combined with a LSTM network. When compared with Stock Chart CNN (SC – CNN) and Stock time series LSTM (ST- LSTM) the CNN + LSTM model performed better with marking profits from investments and also with error scores. The results showed that the

+ LSTM model scored better with metrics like RMSE, RMAE and MAPE by 22.09, 20.89% and 38.17% respectively.

Rezaei et al. [53] proposed an algorithm called Complete Empirical Ensemble Model Decomposition (CEEMD) and also used models like CNN, LSTM, Decision Tree Regressor (DTR), Support Vector Regressor (SVR). The results showed that the hybrid models performed better than vanilla models. More specifically, the RMSE for the CEEMD + CNN + LSTM model was lower compared to CEEMD + LSTM, CNN + LSTM, Vanilla LSTM, SVR and last DTR, providing more promising methods of predicting stock market.

Selvin et al. [54] focused on the short-term aspect of forecasting in which they include models like LSTM, CNN and RNN and linear models like AR, MA, and ARIMA. The purpose of this research is to make a generalized model to forecast in short term scale. The dataset includes stocks from the National Stock Exchange (NSE) like INFOSYS, TCS and Cipla. The deep learning methods showed less RMSE compared to the linear models. The best-performing model for TCS was the RNN model with 3.90 and for Cipla and Infosys stocks the CNN model with error 3.63 and 2.36 respectively.

Pang et al. [55] introduced a modified model called Long short-term memory with automatic encoder based on stock vector (AELSTM) and Long short-term memory neural network with embedded layer (ELSTM). For comparison they also used a Deep Belief Neural network (DBN), DBN + MLP and MLP. In the study all the models embedded the automatic encoder and the embedded layer, and the comparison showed that ALSTM and ELSTM made significant predictions compared to other models and both of them showed accuracy over 50% while the DBN, MLP and DBN + MLP did not pass the 50%.

Zhao et al. [56] proposed a hybrid model called generalized loss CNN – Bidirectional LSTM or (GL-CNN-BiLSTM) and they compared it with binary cross entropy LSTM (BCE – LSTM), GL-LSTM, BCE + LSTM + CNN, CL+CNN+LSTM, BCE + BiLSTM, GL – BiLSTM, BCE + CNN + BiLSTM and GL + CNN + BiLSTM models. The dataset comprises the stocks of the Shanghai, Hong Kong, and NASDAQ Stock Exchanges. The results showed that the CNN + LSTM model performed exceptionally well compared to traditional models using the BCE loss function. The GL + CNN + BiLSTM and GL + CNN + LSTM models were able to make predictions in favor of returns, but not so much in the accuracy of the prediction.

4.3 Other Methods

In order to verify the results from the deep learning methods Theyazn et al. [57] proved that the CNN-LSTM model was performing better compared to the Vanilla LSTM, considering the stock of TESLA, with the R^2 error measurement of the CNN + LSTM being 99.58% and the R^2 of LSTM at 97.20%. On the flip side machine learning algorithms LR, DT and RF are very capable of predicting the stock market trend.

Azaz Hassan Khan and Abdullah Shah [58] proposed 9 ML models like DT, Logistic Regression (LogR), K-Nearest Neighbors (KNN), Naïve Bayes, RF, ADA Boost, SVM, XGBoost and Artificial Neural Network. To measure the effectiveness of the models they used a percentage of their investment returns as a metric. The best performing models where in terms of returns rank as RF with 19.8% and ADA Boost with returns of 15.44 % .

Buacheun [59] faced the problem with another hybrid model compared also with traditional analysis methods. The study used a large number of stocks from 2017 until 2023. The models that were used in the study are LSTM + CNN, CNN + LSTM and CNN + LSTM + Attention Layer (AL) and for the traditional analysis methods they used Stochastic models, Buy & Hold strategy, Relative Strength Index (RSI), Moving Average Convergence Divergence (MACD), and Simple Moving Average (SMA). The conclusions showed that Deep learning models performed better in terms of ROI.

Ming-Tai Wu et al. [60] proposed a new type of deep learning method for forecasting called Stock Sequence Array Convolutional Long Short-Term Model (SACLSTM). In their study, they compared the SACLSTM with models like CNN, SVM, and NN. The forecasting timeline included 1 day, 3 days and 7 days of stock data. The results showed a powerful advantage of the SACLSTM in comparison to LSTM and CNN, in which the accuracy was statistically significant using the SACLSTM.

Qiu et al. [61] analyzed the potential of Attention Model with LSTM network compared with Vanilla LSTM, wavelet transform long-short term model (WLSTM), WLSTM + Attention, and GRU. The dataset consists of the S&P 500, DJIA, and HSI models.

The results showed in the HSI dataset the WLSTM + Attention performed better with an RMSE of 34%, 23% for the S&P 500, and 19% for DIJA.

Lu et al. [62] used a hybrid model of CNN + LSTM in which they compared it with MLP, CNN, RNN, LSTM, and CNN + RNN. The dataset consists of Shanghai Composite Index

from 6/1/1991 until 8/31/2020. The results showed CNN + LSTM outperformed the other models.

Chapter 5. Implementation methodology

This chapter will present the general methodology we have used to approach the stock market predictions. In addition, an introduction to the phases of our methodology will be given, the dataset, the data preprocessing, models and the evaluation metric.

5.1 Machine learning workflow

As shown in (Fig. 19) the systematic steps of creating and developing an environment for data collection and preparation, training, evaluating and deploying models is called machine learning workflow.

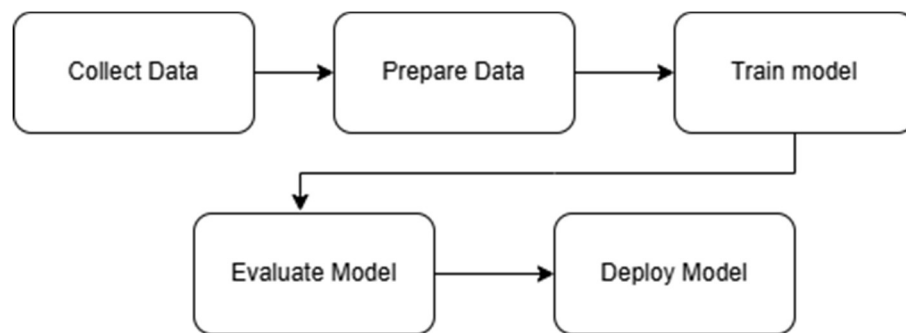


Figure 19. Machine Learning Workflow

In more detail the machine learning workflow includes 3 phases in which the actions are performed to create the optimal model for our specific needs. The first phase is data pre-processing (DRP). This phase contains all the data manipulation operations such as, data cleaning, data transformation or data preprocessing, data splitting and feature extraction to prepare the data to feed our model. The second phase is the learning phase in which first we determine factors that can be detrimental to our model development such as the epochs, batch width, kernels, convolution methods, pooling methods, learning rate etc. Every detail makes an impact in our model accuracy and integrity during this phase. The last phase is the post training phase in which we try to export our model evaluation metrics to check if the training was efficient or not and of course we usually visualize the results of our model to a plot [63].

5.2 Dataset

The dataset we will use consists of stock market prices, including Apple Inc. (AAPL), from 2020-01-01 to 2024-12-31. It includes features such as High, Low, Open, and Close. The High feature indicates the highest price on a specific date, the Low feature shows the lowest

price recorded on a specific date, the Open feature describes the price of the stock when the market opened, and the Close feature indicates the price at which the stock will end by the market's close.

5.3 Data preparation

After the data collection we performed a various data processing to fulfil the specific needs of the models. In detail we used the MinMaxScaler normalization and a times series sequence windows of fixed length (time steps) to pass on the KNN, CNN,LSTM and CNN+LSTM models. The train and test split for the LR model which we split the dataset to 80% training and 20% test sets and lastly the StandartScaler to standardize the features for the SVR model.

5.4 Evaluation metrics

The metrics that have been used to compare the outcomes of ML and NN algorithms is the Root Mean Squared Error (RMSE) [64] (equation 35) also including plotted figures for the neural network models.

$$RMSE = \sqrt{\frac{1}{m} \sum_{i=1}^m |y_i - \bar{y}_i|} \quad (35)$$

5.5 Hardware

The hardware utilized for the thesis consists of:

Operating System Name		Microsoft	Windows	10
		Home		
Version		10.0.19045 Build 19045		
System Type		x64-based PC		
Processor		Intel®	Core™	i7-9750H
		CPU @ 2,60GHz		
Installed	Physical	16.0 GB		
Memory				
GPU		NVIDIA	GeForce	GTX
		1660 Ti		

Table 1, System hardware

5.6 Packages and Frameworks

The packages and frameworks that have been used in this thesis consists of Yahoo Finance (yfinance) library for fetching data purposes, Tensorflow and Keras library, which include the main models that have been used such as: CNN model, LSTM model and MLP model. The machine_learning_models library includes LR, RF, KNN, and SVR. We also used the sklearn library for preprocessing.

5.7 Hyperparameters and models

MODELS	HYPERPARAMETERS/PARAMETERS
LINEAR REGRESSION	-
SUPPORT VECTOR REGRESSOR	Kernel: Linear Regulation Parameter C: 100 Tolerance of termination criterion: 0.2
K-NEAREST NEIGHBOR	K: 5
RANDOM FOREST REGRESSOR	Samples Leaf:1 Sample Split :2 N estimators: 100
MLP	Optimizer: ADAM Loss: MSE Batch Size: 32 Epochs: 100 Input Layer:20 Hidden Layer:20, activation: Sigmoid Hidden Layer:32, activation: Sigmoid Hidden Layer:1, activation: tanh Output Layer: 1 (linear activation)
LSTM	Optimizer: Adam Epochs: 50 Activation: ReLU First Layer: 50 Dropout: 0.2 Second Layer: 60 Dropout 0.3 Third Layer: 80 Dropout:0.4 Forth Layer:120 Dropout:0.5
CNN	Optimizer: Adam Learning Rate: 0.0001 Epochs = 50 Batch Size = 16

CNN + LSTM	Conv1D: Filters=64, Kernel size:3 Activation ReLU MaxPooling1D : pool size:3 Conv1D: Filters:128,Kernel size:3, Activation ReLU MaxPooling1D: pool size:2 Flatten Dense:50 , Activation: ReLU Dropout:0.2
	Conv1D: Filters=64, kernel size :3, activation: ReLU MaxPooling1D: pool_size:2 First Layer:100 Second Layer:50 Dense:50, ReLU Dropout: 0.2 Dense:1 Optimizer: ADAM Learning rate:0.0001 Loss: MSE Epochs:50 Batch Size:16

Table 2. Parameters & Hyperparameters

In order to make the comparison, we needed to reveal the different parameters that we used to make predictions Table 2. Most of the ML models where very limited on parameter tuning, on the other hand the power of NN can give us flexibility and customization. We can notice that the neural network models contain not only the parameters but also the architecture or hyperparameters that we used.

Chapter 6. Results & Conclusions

In this Chapter we will show the different hyperparameters/parameters that we used in each model for a better understanding of the results then we will proceed to the results, and we will compare the results with the help of the RMSE evaluation metric.

6.1 Experimental results

Model	Root Mean Squared Error
Linear Regression	0.96
Support Vector Machine	1.10
K-Nearest Neighbor	3.029
Random Forest	14.26

Table 3. Root Mean Squared Errors for the Machine Learning models

Because of the limitations of each algorithm and the potential influence of parameters on prediction attempts, we can observe differences among the machine learning models. Better or worse, the model's ability to make predictions increases with number of parameters it contains. We can observe that accuracy decreases with the number of parameters set to default values. Table 3 shows that the model with the most parameters (RF) performed the worst out of all the models, while the Linear Regression model with no parameters at all won. Although the RF's parameters were adjusted, the results were neither ideal nor precise.

Model	Root Mean Squared Error
MLP	5.25
LSTM	10.06
CNN	8.42
CNN + LSTM	4.96

Table 4. Root Mean Squared Errors for the Neural Network models

The same scenario appears also about the Neural Network models in some cases. The winner is the CNN + LSTM model which is the most utilized method for predicting the stock market as mentioned on related work section. The MLP was second and for the same reasons the MLP model has the lowest hyperparameter and parameters compared to other tree models, this was an advantage and without any optimization made better predictions from CNN and LSTM models on vanilla tuning. The hyperparameter tuning involves the architecture of the model, how many hidden layers does it contain, how many nodes each layer contains, what is the Batch size and so on. The difference between the CNN and LSTM is that the CNN model does not have complicated layers like the LSTM and makes the predictions more accurate

(Convolutional layer and MaxPooling layers). We see that the LSTM (Fig.20) did a lot of fluctuations compared to the chart and in the end was late on the upgoing trend on head and shoulders pattern that appeared. The CNN model (Fig.21) may be classified as a late trend predictor because it acquired the correct trend later than the actual values. The MLP correctly predicted the trend (Fig.19) and only three times did it produce a lower or higher peak than the actual one. Overall, we can observe that the CNN and LSTM (Fig.22) are nearly identical to the real prices, and we can see a more accurate price prediction that does not overfit.

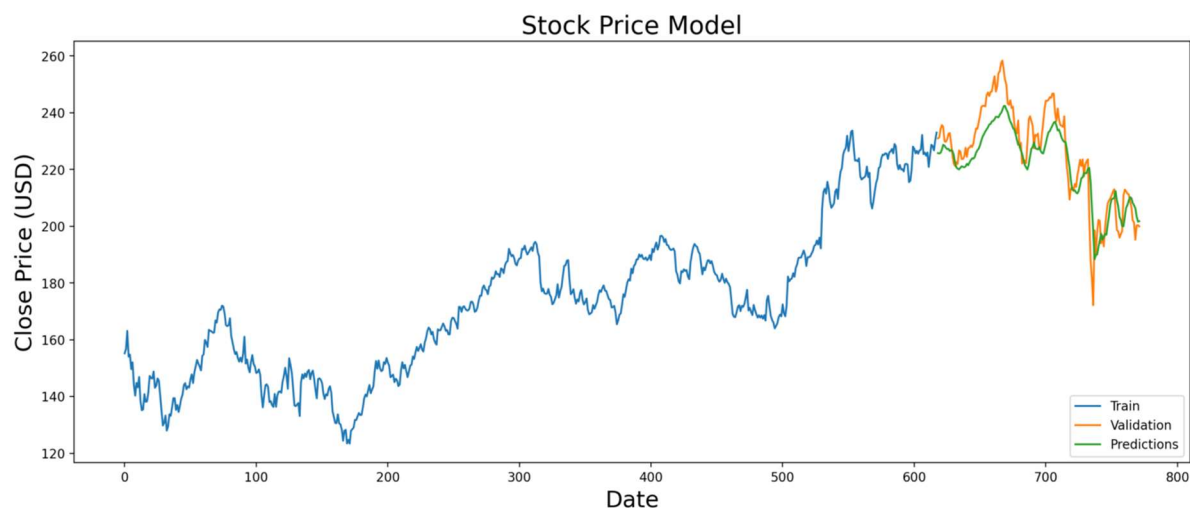


Figure 20. Actual vs Predicted values from MLP model

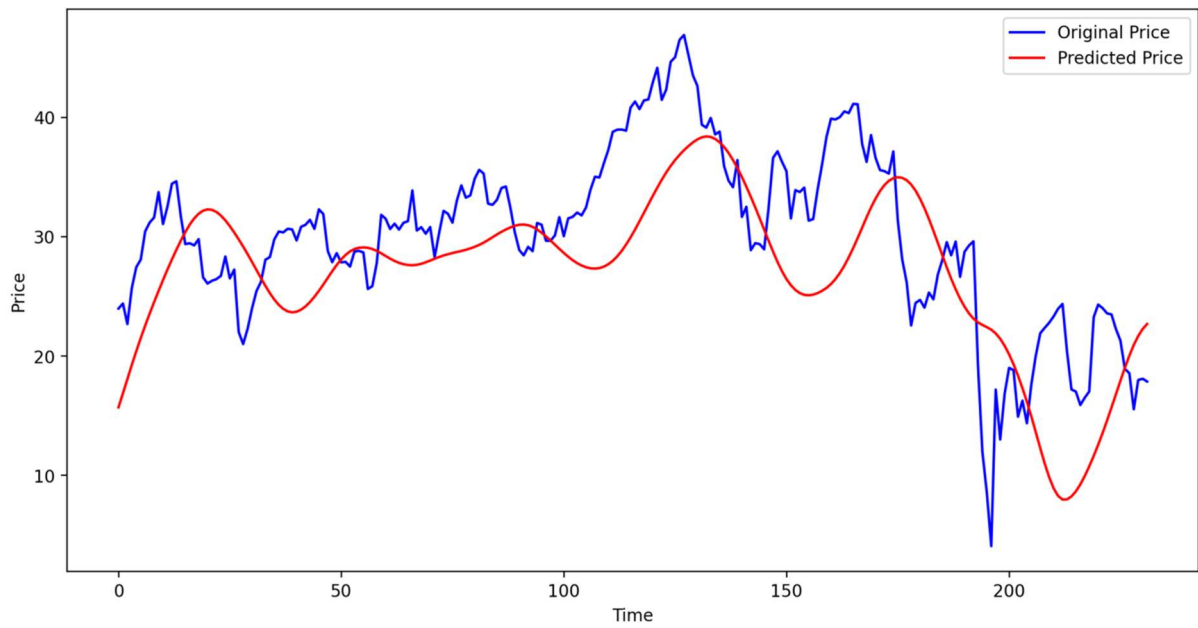


Figure 21. Actual vs Predicted values from LSTM model

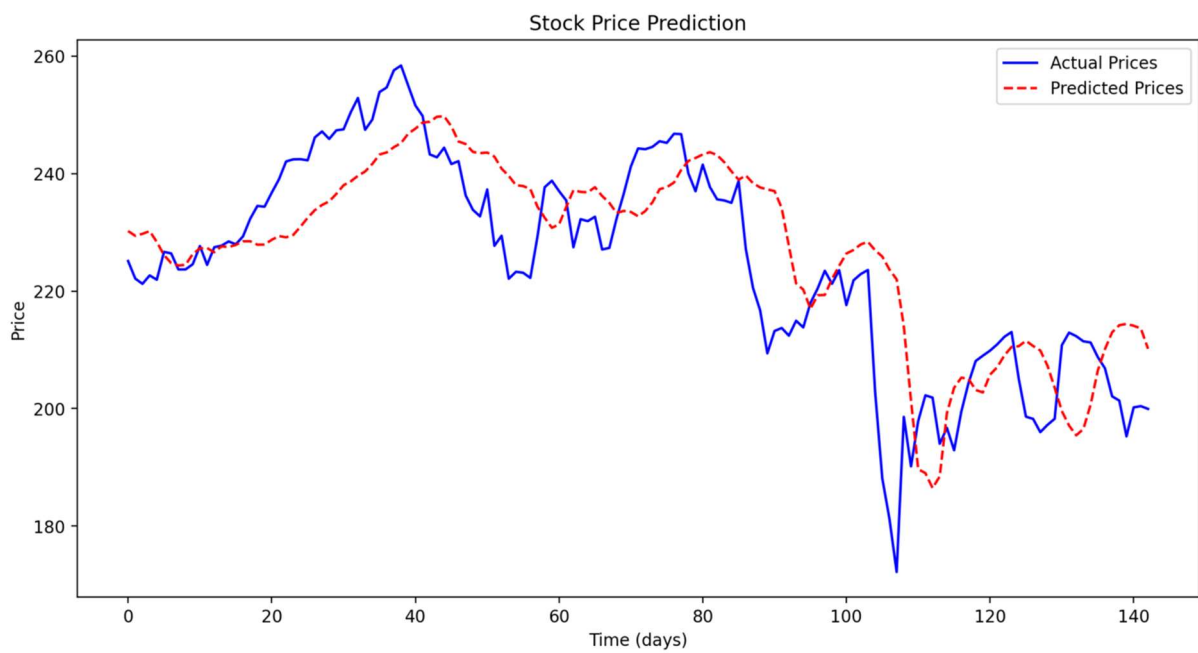


Figure 22. Actual vs Predicted values from CNN model

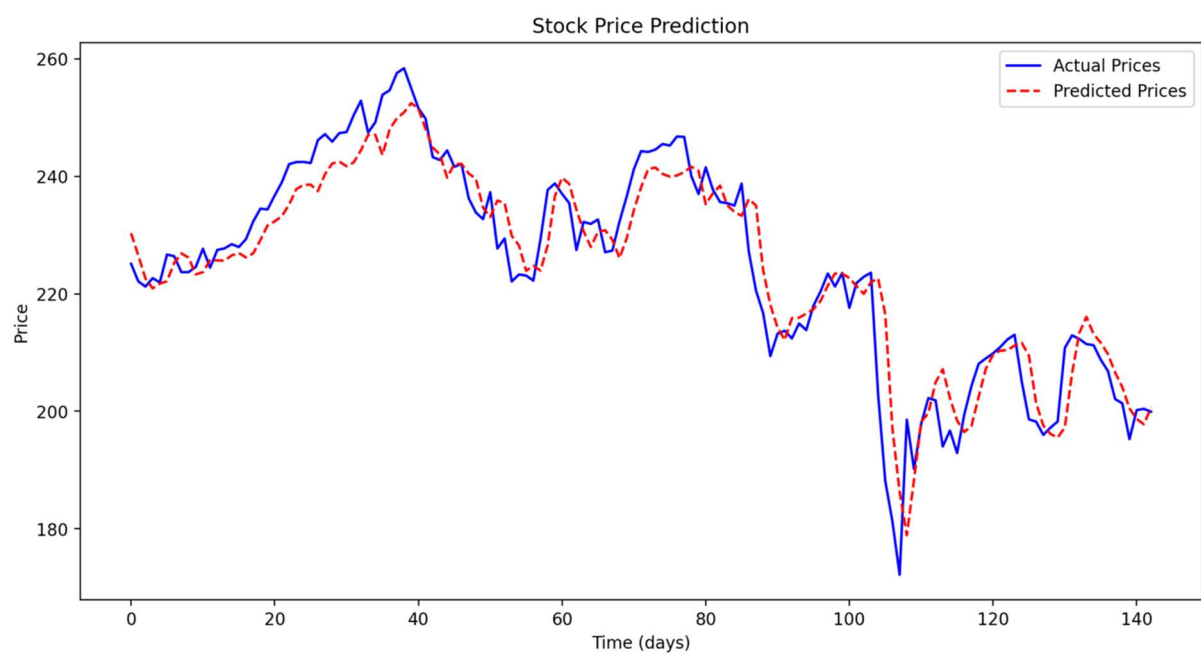


Figure 23. Actual vs Predicted values from CNN model

Chapter 7. Conclusions and future work

Stock market predictions are a complex task themselves, involving a variety of variables and metrics that go beyond those explored in this thesis. Human nature also tends to influence the trends, making it impossible to predict future prices or trends without information. The machine learning algorithms conducted in this thesis were among the most selected in stock price prediction. More sophisticated models appeared to make the predictions more accurate and over a longer timeframe in the future. The correct or wrong usage of the parameters, hyperparameters, and understanding of the algorithms can make or break the models, especially in the stock market prediction problem. Although simple machine learning algorithms predict the stock market with less error, with a warning of overfitting, the neural networks have more potential in predicting future prices with optimal tuning without overfitting. We used the default values to see the difference between the ML and NN methods. In conclusion, the best stock predictor from the ML models was the LR model with the lowest RMSE (0.94), and from the neural network models was the CNN+LSTM model with the lowest RMSE (4.96), making the best predictions of its category.

One area of further work could be optimization of parameters and hyperparameters, including also more complex architectures of neural networks. More specifically, generative artificial intelligence could be useful as a feature extractor and include the news that influences stock market prices through scraping methods from websites. The outcome will influence the projection of the predictions and will help financial analysts and investors to make more accurate predictions and not only with quantitative data but also qualitative metrics that can help throughout a stock portfolio. In general, many architectures are published that involve complex combinations of algorithms with different hyperparameters and parameter tuning, increasing the area in which a stock market prediction model can perform. After the models are properly trained with a normal percentage of RMSE we could use a “rolling technique” to make a future prediction in our stocks of choice and also, we could train even further our models with index funds to make more accurate predictions in a large scale datasets. The modern usage of artificial intelligence has become necessary for all cognitive and analytical tasks and without it, we could lose valuable insight in our daily lives but even more importantly on the stock predictions.

Bibliography & References

- [1] Rjumohan, “Munich Personal RePEc Archive Stock Markets: An Overview and A Literature Review,” 2019.
- [2] M. Nabipour, P. Nayyeri, H. Jabani, S. S., and A. Mosavi, “Predicting Stock Market Trends Using Machine Learning and Deep Learning Algorithms Via Continuous and Binary Data; a Comparative Analysis,” *IEEE Access*, vol. 8, pp. 150199–150212, 2020, doi: 10.1109/ACCESS.2020.3015966.
- [3] A. M. Turing, “M I N D A QUARTERLY REVIEW OF PSYCHOLOGY AND PHILOSOPHY I-COMPUTING MACHINERY AND INTELLIGENCE,” 1950. [Online]. Available: <https://academic.oup.com/mind/article/LIX/236/433/986238>
- [4] S. Kakad, V. Kale, A. Kakare, A. Kalokhe, and A. Yewale, “International Conference on Communication and Information Processing The Evolution of Computers,” 2023. [Online]. Available: <https://ssrn.com/abstract=4687940>
- [5] M. Ballings, D. Van Den Poel, N. Hespeels, and R. Gryp, “Evaluating multiple classifiers for stock price direction prediction,” *Expert Syst Appl*, vol. 42, no. 20, pp. 7046–7056, Jun. 2015, doi: 10.1016/j.eswa.2015.05.013.
- [6] A. Abraham, B. Nath, and P. K. Mahanti, “Hybrid intelligent systems for stock market analysis,” in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Springer Verlag, 2001, pp. 337–345. doi: 10.1007/3-540-45718-6_38.
- [7] Y. Xu *et al.*, “Artificial intelligence: A powerful paradigm for scientific research,” Nov. 28, 2021, *Cell Press*. doi: 10.1016/j.xinn.2021.100179.
- [8] S. . Samoili, M. . López Cobo, E. . Gómez, G. . De Prato, F. . Martínez-Plumed, and B. . Delipetrev, *AI watch : defining Artificial Intelligence : towards an operational definition and taxonomy of artificial intelligence*. Publications Office of the European Union, 2020.
- [9] Manning Christopher, “AI-Definitions-HAI,” *H.A.I.*, p. 1, Sep. 2020, Accessed: Feb. 05, 2025. [Online]. Available: <https://hai.stanford.edu/sites/default/files/2020-09/AI-Definitions-HAI.pdf>
- [10] D. Muhammad, I. Ahmed, K. Naveed, and M. Bendechache, “An explainable deep learning approach for stock market trend prediction,” *Heliyon*, vol. 10, no. 21, Nov. 2024, doi: 10.1016/j.heliyon.2024.e40095.
- [11] P. Tupe-Waghmare, “Prediction of Stocks and Stock Price using Artificial Intelligence : Prediction of Stocks and Stock Price using Artificial Intelligence : A Bibliometric Study using Scopus Database A Bibliometric Study using Scopus Database Prediction of Stocks and Stock Price using Artificial Intelligence: A Bibliometric Study using Scopus Database,” 2021.

- [12] P. D. Yoo, M. H. Kim, and T. Jan, "Machine learning techniques and use of event information for stock market prediction: A survey and evaluation," in *Proceedings - International Conference on Computational Intelligence for Modelling, Control and Automation, CIMCA 2005 and International Conference on Intelligent Agents, Web Technologies and Internet*, 2005, pp. 835–841. doi: 10.1109/cimca.2005.1631572.
- [13] K. P. . Murphy, *Machine learning : a probabilistic perspective*. MIT Press, 2012.
- [14] M. T. Almuqati, F. Sidi, S. N. M. Rum, M. Zolkepli, and I. Ishak, "Challenges in Supervised and Unsupervised Learning: A Comprehensive Overview," *Int J Adv Sci Eng Inf Technol*, vol. 14, no. 4, pp. 1449–1455, 2024, doi: 10.18517/ijaseit.14.4.20191.
- [15] A. Jung, "Machine Learning: The Basics," 2023.
- [16] Ulle Endriss, "Institute for Logic, Language and Computation," Amsterdam, Sep. 2014. Accessed: Feb. 05, 2025. [Online]. Available: <https://ocw.upj.ac.id/files/Textbook-TIF212-Prolog-Tutorial-3.pdf>
- [17] L. M. Augusto, "Deductive Computing over Knowledge Bases: Prolog and Datalog," 2024.
- [18] J. Kontos, "Human-System Interface with Explanation of Actions for Autonomous Anti-UAV Systems," *International Journal of Artificial Intelligence & Applications*, vol. 12, no. 04, pp. 51–58, Jul. 2021, doi: 10.5121/ijaia.2021.12404.
- [19] V. Tabakova-Komsalova, I. Stoyanov, G. Cholakov, and M. Maglizhanova, "Introducing Artificial Intelligence to Secondary Schools Through STEM Learning and the Logic Programming Language Prolog," *TEM Journal*, vol. 13, no. 4, pp. 3221–3230, Nov. 2024, doi: 10.18421/TEM134-56.
- [20] A. Lally and P. Fodor, "Natural Language Processing With Prolog in the IBM Watson System," 2011.
- [21] C. C. Aggarwal, "A Textbook Second Edition Neural Networks and Deep Learning."
- [22] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, "Linear Regression," 2023, pp. 69–134. doi: 10.1007/978-3-031-38747-0_3.
- [23] H. Blockeel, L. Devos, B. Frénay, G. Nanfack, and S. Nijssen, "Decision trees: from efficient prediction to responsible AI," 2023, *Frontiers Media SA*. doi: 10.3389/frai.2023.1124553.
- [24] G. Biau and G. B. Fr, "Analysis of a Random Forests Model," 2012.
- [25] S. Mutasa, S. Sun, and R. Ha, "Understanding artificial intelligence based radiology studies: What is overfitting?," Sep. 01, 2020, *Elsevier Inc*. doi: 10.1016/j.clinimag.2020.04.025.
- [26] Andreas Wålinder, "Examensarbete Evaluation of logistic regression and random forest classification based on prediction accuracy and metadata analysis," 391 82 Kalmar | 351 95 Växjö, Jun. 2014.

- [27] G. Biau and E. Scornet, “A Random Forest Guided Tour.” [Online]. Available: <http://www.kaggle.com/c/dsg-hackathon>
- [28] M. Andrychowicz *et al.*, “Learning to learn by gradient descent by gradient descent,” Jun. 2016, [Online]. Available: <http://arxiv.org/abs/1606.04474>
- [29] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, “A Comparative Analysis of XGBoost,” Nov. 2019, doi: 10.1007/s10462-020-09896-5.
- [30] D. Nielsen, “Tree Boosting With XGBoost Why Does XGBoost Win ‘Every’ Machine Learning Competition?”
- [31] S. S. Dhaliwal, A. Al Nahid, and R. Abbas, “Effective intrusion detection system using XGBoost,” *Information (Switzerland)*, vol. 9, no. 7, Jun. 2018, doi: 10.3390/info9070149.
- [32] R. Wang, J. Zhang, B. Shan, M. He, and J. Xu, “XGBoost Machine Learning Algorithm for Prediction of Outcome in Aneurysmal Subarachnoid Hemorrhage,” *Neuropsychiatr Dis Treat*, vol. 18, pp. 659–667, 2022, doi: 10.2147/NDT.S349956.
- [33] V. Jakkula, “Tutorial on Support Vector Machine (SVM).” Accessed: Mar. 12, 2025. [Online]. Available: <https://course.ccs.neu.edu/cs5100f11/resources/jakkula.pdf>
- [34] Nello. Cristianini and John. Shawe-Taylor, *An Introduction to support vector machines : and other kernel-based learning methods*. Cambridge University Press, 20142004.
- [35] F. Rosenblatt, “THE PERCEPTRON: A PROBABILISTIC MODEL FOR INFORMATION STORAGE AND ORGANIZATION IN THE BRAIN 1.”
- [36] K. L. Du, C. S. Leung, W. H. Mow, and M. N. S. Swamy, “Perceptron: Learning, Generalization, Model Selection, Fault Tolerance, and Role in the Deep Learning Era,” Dec. 01, 2022, *MDPI*. doi: 10.3390/math10244730.
- [37] J. Waldo, “A comparative study of back propagation and its alternatives on multilayer perceptrons,” May 2022, [Online]. Available: <http://arxiv.org/abs/2206.06098>
- [38] G. Capobianco *et al.*, “Image convolution: a linear programming approach for filters design,” *Soft comput*, vol. 25, no. 14, pp. 8941–8956, Jul. 2021, doi: 10.1007/s00500-021-05783-5.
- [39] H. Gholamalinezhad and H. Khosravi, “Pooling Methods in Deep Neural Networks, a Review.”
- [40] Z. Li, S. H. Wang, R. R. Fan, G. Cao, Y. D. Zhang, and T. Guo, “Teeth category classification via seven-layer deep convolutional neural network with max pooling and global average pooling,” *Int J Imaging Syst Technol*, vol. 29, no. 4, pp. 577–583, Dec. 2019, doi: 10.1002/ima.22337.
- [41] S. Sharma, S. Sharma, and A. Athaiya, “ACTIVATION FUNCTIONS IN NEURAL NETWORKS,” 2020. [Online]. Available: <http://www.ijeast.com>

- [42] T. A. Kalaycı and U. Asan, “Improving Classification Performance of Fully Connected Layers by Fuzzy Clustering in Transformed Feature Space,” *Symmetry (Basel)*, vol. 14, no. 4, Apr. 2022, doi: 10.3390/sym14040658.
- [43] S. Hochreiter and J. “Urgen Schmidhuber, “Long Short-Term Memory.”
- [44] M. Usmani, Z. A. Memon, A. Zulfiqar, and R. Qureshi, “Preptimize: Automation of Time Series Data Preprocessing and Forecasting,” *Algorithms*, vol. 17, no. 8, Aug. 2024, doi: 10.3390/a17080332.
- [45] Y. Zhao, “Stock Price Prediction Based on Linear Regression,” 2025, doi: 10.54254/2753-8818/84/2025.21203.
- [46] B. Panwar, G. Dhuriya, P. Johri, S. Singh Yadav, and N. Gaur, “Stock Market Prediction Using Linear Regression and SVM,” in *2021 International Conference on Advance Computing and Innovative Technologies in Engineering (ICACITE)*, IEEE, 2021, pp. 629–631. doi: 10.1109/ICACITE51222.2021.9404733.
- [47] M. Biswas, A. Shome, Md. A. Islam, A. J. Nova, and S. Ahmed, “Predicting Stock Market Price: A Logical Strategy using Deep Learning,” in *2021 IEEE 11th IEEE Symposium on Computer Applications & Industrial Electronics (ISCAIE)*, IEEE, 2021, pp. 218–223. doi: 10.1109/ISCAIE51753.2021.9431817.
- [48] O. Kelany, S. Aly, and M. A. Ismail, “Deep Learning Model for Financial Time Series Prediction,” in *2020 14th International Conference on Innovations in Information Technology (IIT)*, IEEE, 2020, pp. 120–125. doi: 10.1109/IIT50501.2020.9299063.
- [49] B. M. Henrique, V. A. Sobreiro, and H. Kimura, “Stock price prediction using support vector regression on daily and up to the minute prices,” *Journal of Finance and Data Science*, vol. 4, no. 3, pp. 183–201, Sep. 2018, doi: 10.1016/j.jfds.2018.04.003.
- [50] S. Jin, “A Comparative Analysis of Traditional and Machine Learning Methods in Forecasting the Stock Markets of China and the US,” 2024. [Online]. Available: www.ijacsa.thesai.org
- [51] Y. Wu, Z. Fu, ·Xiaoxuan Liu, and ·Yuan Bing, “A hybrid stock market prediction model based on GNG and reinforcement learning,” *Expert Syst Appl*, vol. 228, Oct. 2023, doi: 10.1016/j.eswa.2023.120474.
- [52] T. Kim and H. Y. Kim, “Forecasting stock prices with a feature fusion LSTM-CNN model using different representations of the same data,” *PLoS One*, vol. 14, no. 2, p. e0212320, 2019, doi: 10.1371/journal.pone.0212320.
- [53] H. Rezaei, H. Faaljoui, and G. Mansourfar, “Stock price prediction using deep learning and frequency decomposition,” *Expert Syst Appl*, vol. 169, p. 114332, 2021, doi: 10.1016/j.eswa.2020.114332.
- [54] S. Selvin, R. Vinayakumar, E. A. Gopalakrishnan, V. K. Menon, and K. P. Soman, “Stock price prediction using LSTM, RNN and CNN-sliding window model,” in *2017*

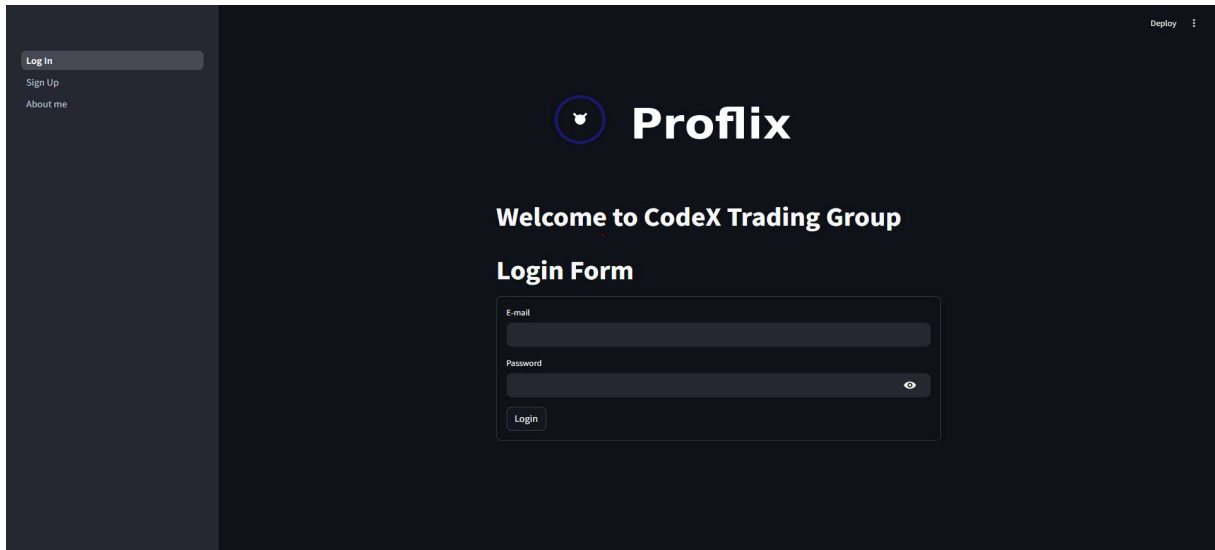
International Conference on Advances in Computing, Communications and Informatics (ICACCI), IEEE, 2017, pp. 1643–1647. doi: 10.1109/ICACCI.2017.8126078.

- [55] X. Pang, Y. Zhou, P. Wang, W. Lin, and V. Chang, “An innovative neural network approach for stock market prediction,” *Journal of Supercomputing*, vol. 76, no. 3, pp. 2098–2118, Mar. 2020, doi: 10.1007/s11227-017-2228-y.
- [56] X. Zhao, Y. Liu, and Q. Zhao, “Generalized Loss-Based CNN-BiLSTM for Stock Market Prediction,” *International Journal of Financial Studies*, vol. 12, no. 3, p. 61, Jun. 2024, doi: 10.3390/ijfs12030061.
- [57] T. H. H. Aldhyani and A. Alzahrani, “Framework for Predicting and Modeling Stock Market Prices Based on Deep Learning Algorithms,” *Electronics (Basel)*, vol. 11, no. 19, p. 3149, 2022, doi: 10.3390/electronics11193149.
- [58] A. H. Khan *et al.*, “A performance comparison of machine learning models for stock market prediction with novel investment strategy,” *PLoS One*, vol. 18, no. 9 September, Sep. 2023, doi: 10.1371/journal.pone.0286362.
- [59] W. Buachuen and P. Kantavat, “Automated Stock Trading System using Technical Analysis and Deep Learning Models,” in *Proceedings of the 13th International Conference on Advances in Information Technology*, New York, NY, USA: ACM, Dec. 2023, pp. 1–9. doi: 10.1145/3628454.3631670.
- [60] J. M.-T. Wu, Z. Li, N. Herencsar, B. Vo, and J. C.-W. Lin, “A graph-based CNN-LSTM stock price prediction algorithm with leading indicators,” *Multimed Syst*, vol. 29, no. 3, pp. 1751–1770, 2023, doi: 10.1007/s00530-021-00758-w.
- [61] J. Qiu, B. Wang, and C. Zhou, “Forecasting stock prices with long-short term memory neural network based on attention mechanism,” *PLoS One*, vol. 15, no. 1, Jan. 2020, doi: 10.1371/journal.pone.0227222.
- [62] W. Lu, J. Li, Y. Li, A. Sun, and J. Wang, “A CNN-LSTM-based model to forecast stock prices,” *Complexity*, vol. 2020, 2020, doi: 10.1155/2020/6622927.
- [63] D. Xin, L. Ma, S. Song, and A. Parameswaran, “How Developers Iterate on Machine Learning Workflows -- A Survey of the Applied Machine Learning Literature,” Mar. 2018, [Online]. Available: <http://arxiv.org/abs/1803.10311>
- [64] Y. Hong, “Boosting the Accuracy of Stock Market Prediction via Multi-Layer Hybrid MTL Structure,” Jan. 2025, [Online]. Available: <http://arxiv.org/abs/2501.09760>
- [65] <https://appliedmachinelearning.wordpress.com/2017/03/09/understanding-support-vector-machines-a-primer/>
- [66] https://miro.medium.com/v2/resize:fit:720/format:webp/1*2UV8DrF8wbE7PLiYiSW5.png
- [67] <https://tex.stackexchange.com/questions/503883/illustrating-the-random-forest-algorithm-in-tikz>

- [68] <https://www.researchgate.net/profile/Arief-Muharram/publication/362542533/figure/fig1/AS:11431281195672126@1696435738568/enn-diagram-schematic-representation-of-the-hierarchy-of-terms-used-in-artificial>

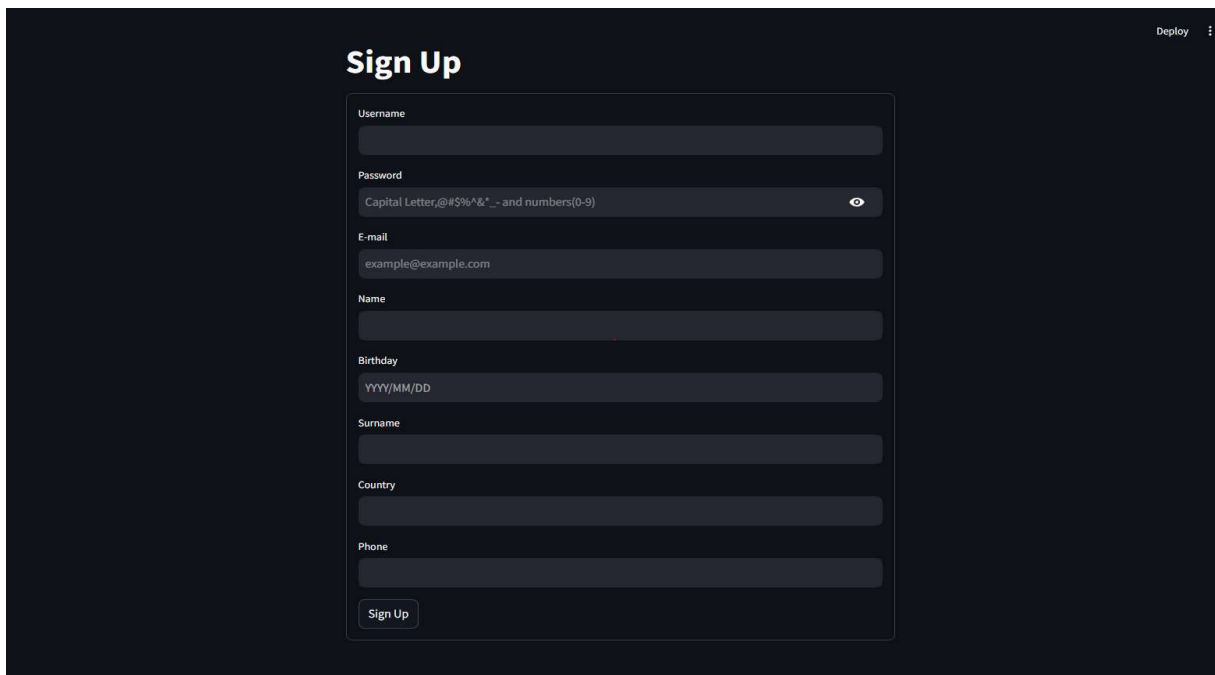
Appendix

Using Python for creating and deploying the models, we used a library called streamlit to create the User interface for the user to interact with the models and to set the dataset to its desired dataset.

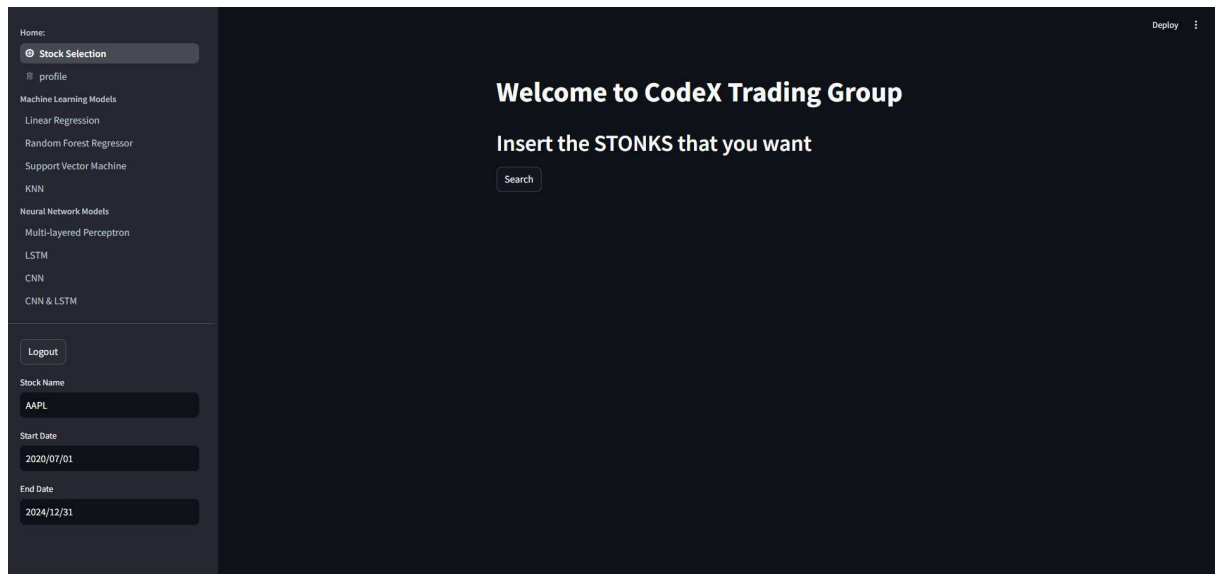


The screenshot shows the login page of the Proflix application. The page has a dark blue background. On the left side, there is a vertical sidebar with links: "Log In", "Sign Up", and "About me". The "Log In" link is highlighted. In the center of the page, there is a logo for "Proflix" which consists of a blue circle with a white bird-like icon inside, followed by the word "Proflix" in white. Below the logo, the text "Welcome to CodeX Trading Group" is displayed in white. Underneath that, the heading "Login Form" is shown. The login form itself is a white box with two input fields: "E-mail" and "Password". The "Password" field has a toggle icon (an eye) to its right. Below the input fields is a "Login" button. In the top right corner of the page, there is a "Deploy" link with a dropdown arrow.

Here we can login as a use or we can sign up from Sign up page



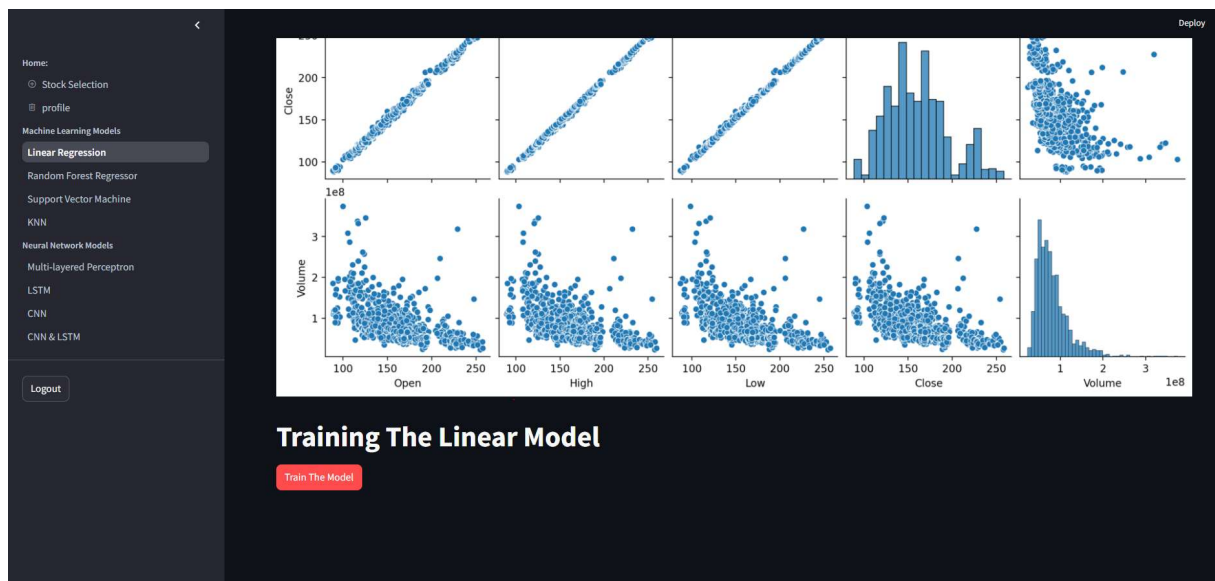
The screenshot shows the sign up page of the Proflix application. The page has a dark blue background. In the top right corner, there is a "Deploy" link with a dropdown arrow. The heading "Sign Up" is prominently displayed in white. Below the heading is a white box containing the sign up form. The form has several input fields: "Username", "Password" (with a password strength indicator below it: "Capital Letter,@#\$%^&* _ - and numbers(0-9)"), "E-mail" (with the example "example@example.com"), "Name", "Birthday" (with a date format "YYYY/MM/DD"), "Surname", "Country", and "Phone". At the bottom of the form is a "Sign Up" button.



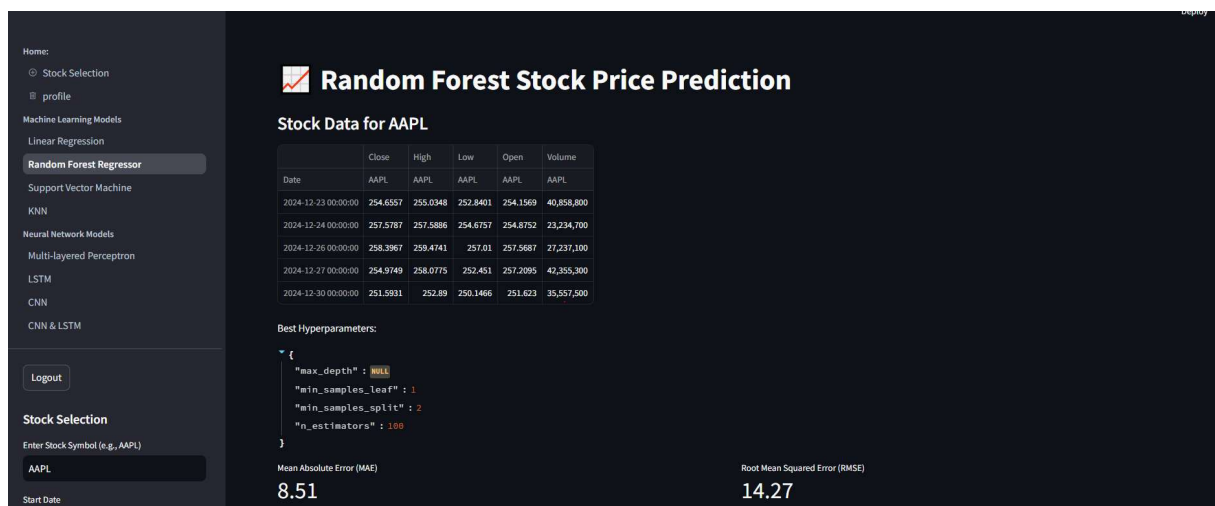
After we finish the login we can choose the stock that we want from the panel on the left (Starting date and End date)



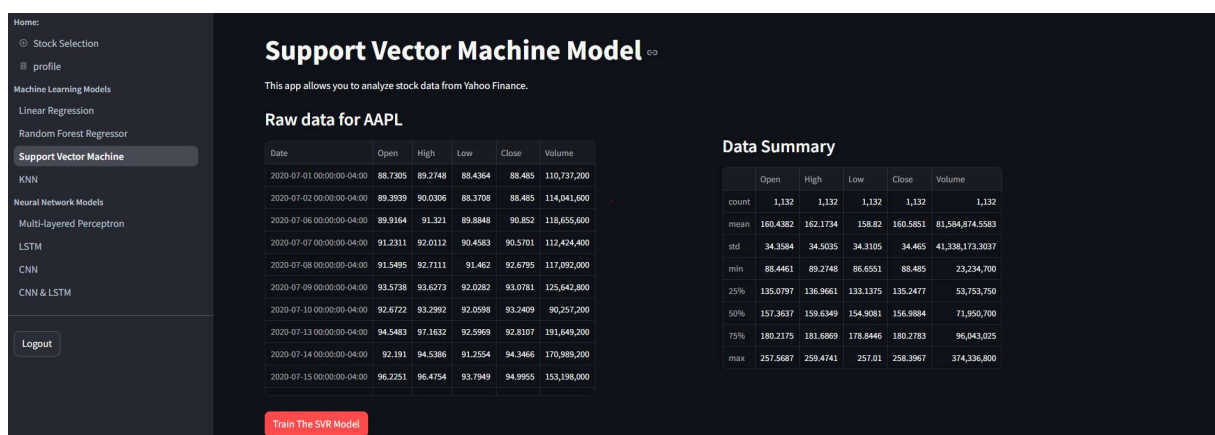
Here we can visualize the stock that we desire



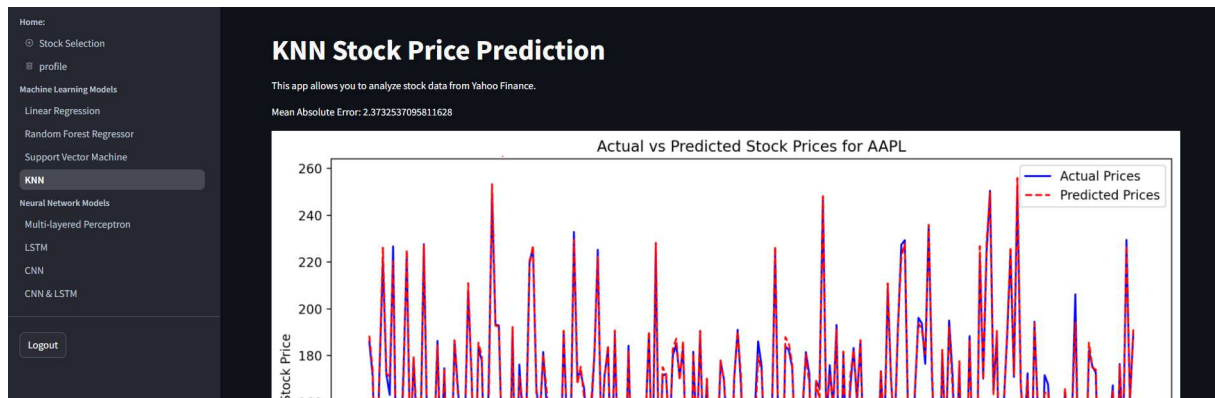
After we choose the stock that we desire we can train the model with the linear regression



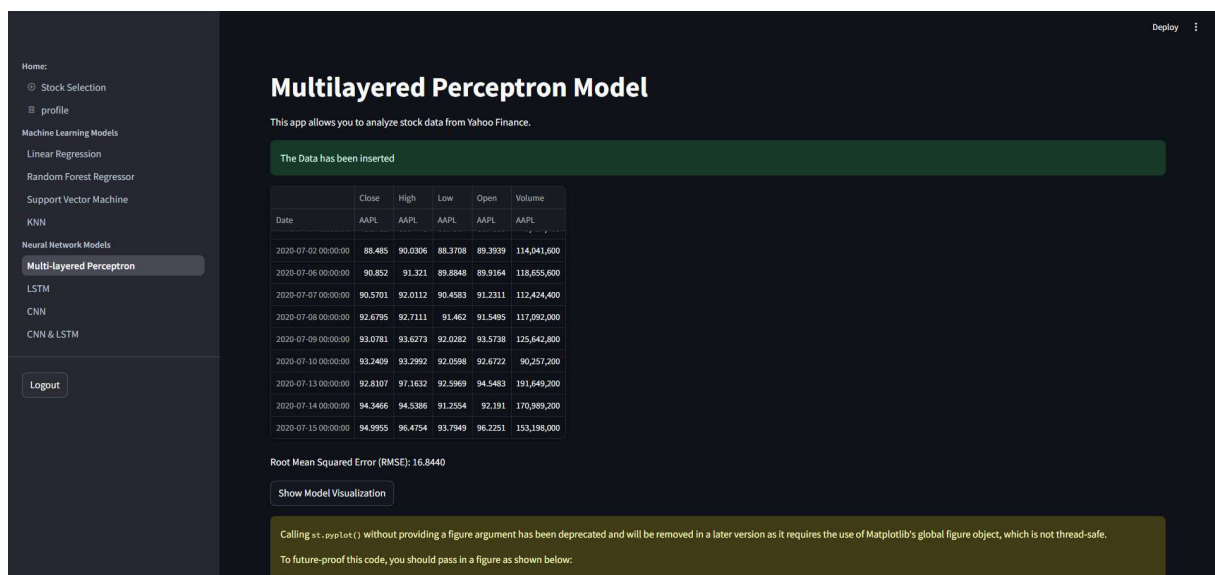
We can train here with the random forest regressor



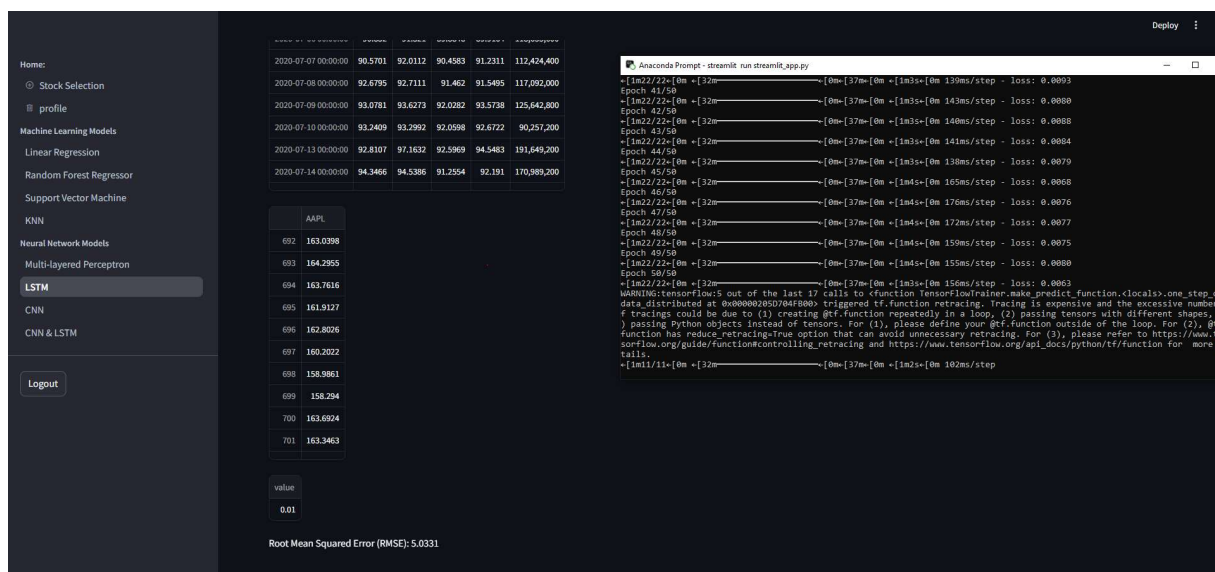
We can train here with the Support vector regressor



We can train here with the KNN model



We can train an MLP model



We can train an LSTM model

Home:

Stock Selection

profile

Machine Learning Models

Linear Regression

Random Forest Regressor

Support Vector Machine

KNN

Neural Network Models

Multi-layered Perceptron

LSTM

CNN

CNN & LSTM

Logout

Deploy

Convolutional Neural Network

This app allows you to analyze stock data from Yahoo Finance.

The Data has been inserted

	Close	High	Low	Open	Volume
Date	AAPL	AAPL	AAPL	AAPL	AAPL
2020-07-01 00:00:00	88.485	89.2748	88.4364	88.7305	110,737,200
2020-07-02 00:00:00	88.485	90.0306	88.3708	89.3939	114,041,600
2020-07-06 00:00:00	90.852	91.321	89.8848	89.9164	118,655,600
2020-07-07 00:00:00	90.5701	92.0112	90.4583	91.2311	112,424,400
2020-07-08 00:00:00	92.6795	92.7111	91.462	91.5495	117,092,000
2020-07-09 00:00:00	93.0781	93.6273	92.0282	93.5738	125,642,800
2020-07-10 00:00:00	93.2409	93.2992	92.0598	92.6722	90,257,200
2020-07-13 00:00:00	92.8107	97.1632	92.5969	94.5483	191,649,200
2020-07-14 00:00:00	94.3466	94.5386	91.2354	92.191	170,989,200

Stock Price Prediction

Root Mean Squared Error (RMSE): 7.2346

Stock Price Prediction

```

[0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0015 - val_loss: 0.0023
Epoch 35/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [7ms/step - loss: 0.0018 - val_loss: 0.0013
Epoch 36/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0014 - val_loss: 0.0017
Epoch 37/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0017 - val_loss: 0.0055
Epoch 38/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [7ms/step - loss: 0.0010 - val_loss: 0.0047
Epoch 39/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0013 - val_loss: 0.0024
Epoch 40/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0013 - val_loss: 0.0037
Epoch 41/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [7ms/step - loss: 0.0013 - val_loss: 0.0055
Epoch 42/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0014 - val_loss: 0.0017
Epoch 43/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [7ms/step - loss: 0.0013 - val_loss: 0.0028
Epoch 44/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [7ms/step - loss: 0.0012 - val_loss: 0.0056
Epoch 45/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0014 - val_loss: 0.0012
Epoch 46/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0016 - val_loss: 0.0012
Epoch 47/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0015 - val_loss: 0.0037
Epoch 48/50
[1m0s-[54m-[0m- [32m- [0m- [37m-[0m- [1m0s-[0m- [0ms/step - loss: 0.0013 - val_loss: 0.0035
Epoch 49/50

```

We can train the CNN model

Home:

Stock Selection

profile

Machine Learning Models

Linear Regression

Random Forest Regressor

Support Vector Machine

KNN

Neural Network Models

Multi-layered Perceptron

LSTM

CNN

CNN & LSTM

Logout

Deploy

CNN + LSTM Model

This app allows you to analyze stock data from Yahoo Finance.

The Data has been inserted

	Close	High	Low	Open	Volume
Date	AAPL	AAPL	AAPL	AAPL	AAPL
2020-07-01 00:00:00	88.485	89.2748	88.4364	88.7305	110,737,200
2020-07-02 00:00:00	88.485	90.0306	88.3708	89.3939	114,041,600
2020-07-06 00:00:00	90.852	91.321	89.8848	89.9164	118,655,600
2020-07-07 00:00:00	90.5701	92.0112	90.4583	91.2311	112,424,400
2020-07-08 00:00:00	92.6795	92.7111	91.462	91.5495	117,092,000
2020-07-09 00:00:00	93.0781	93.6273	92.0282	93.5738	125,642,800
2020-07-10 00:00:00	93.2409	93.2992	92.0598	92.6722	90,257,200
2020-07-13 00:00:00	92.8107	97.1632	92.5969	94.5483	191,649,200
2020-07-14 00:00:00	94.3466	94.5386	91.2354	92.191	170,989,200

Stock Price Prediction

Root Mean Squared Error (RMSE): 9.1537

Stock Price Prediction

We can train the CNN + LSTM model

52